Introduzione all'apprendimento automatico

Mattia Masina
CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1 Introduzione all'apprendimento automatico **3**

1.1 Introduzione al Machine Learning . . . 3
1.1.1 Che cos'è il Machine Learning . . . 3
1.1.2 Le feature e il Deep Learning . . . 4
1.2 Alberi di decisione . . . 6
1.2.1 Apprendimento, struttura ed espressività . . . 6
1.2.2 Costruzione top-down, entropia e attributi continui . . . 8
1.2.3 Overfitting, potatura e impurità di Gini . . . 12
1.2.4 Pregi, difetti, Random Forest e riepilogo . . . 13
1.3 Il paradigma probabilistico . . . 14
1.3.1 Basi di probabilità . . . 14
1.3.2 Probabilità condizionata e indipendenza . . . 16
1.3.3 Teorema di Bayes . . . 17
1.3.4 La distribuzione congiunta . . . 18
1.4 Naïve Bayes . . . 19
1.4.1 Ipotesi di indipendenza, classificazione ed esempio . . . 19
1.4.2 Natura generativa, limiti e stima MLE . . . 22
1.4.3 Testi, prodotto scalare e linearità di Naïve Bayes . . . 26
1.4.4 Attributi continui e valutazione del modello . . . 30
1.5 Regressione Logistica . . . 36
1.5.1 Dall'approccio generativo a quello discriminativo e funzione logistica . . . 37
1.5.2 Addestramento e discesa del gradiente . . . 40
1.5.3 Regolarizzazione e confronto con i modelli generativi . . . 46
1.5.4 Modelli lineari in alta dimensione e multiclasse . . . 48

2 Reti neurali e deep learning **51**

2.1 Deep Learning in breve . . . 51
2.1.1 Che cos'è il Deep Learning e dove si applica . . . 51
2.1.2 La tecnica di base: le reti neurali . . . 52
2.2 Le reti neurali . . . 53
2.2.1 Neurone artificiale, funzioni di attivazione e confronto col cervello . . . 53
2.3 Topologie di rete e tipi di strato . . . 57
2.3.1 Feed-forward, strati densi e strati convolutivi . . . 57
2.4 Tuffo nel Deep Learning . . . 60
2.4.1 Obiettivi, strumenti e storia . . . 60

1 Introduzione all'apprendimento automatico

1.1 Introduzione al Machine Learning

1.1.1 Che cos'è il Machine Learning

Nell'informatica classica il programmatore scrive le regole che trasformano l'input nell'output. Per molti problemi reali, però, queste regole sono quasi impossibili da scrivere a mano: pensiamo a riconoscere un volto in una foto o a capire se una mail è spam. Sono i problemi tipici del Machine Learning: **classificazione** (spam, sentiment analysis, frodi), **riconoscimento di immagini**, riconoscimento vocale e musicale, **anomaly detection**, **clustering** e **AI generativa**.
Questi domini hanno alcune caratteristiche in comune:

- **poca conoscenza teorica**: non esiste una teoria che descriva il fenomeno in modo esatto;
- **tante feature in input**: ogni dato è descritto da moltissimi valori (es. i pixel di un'immagine);
- **tanti dati**: servono moltissimi esempi (ImageNet contiene circa 10 milioni di immagini);
- **adattabilità**: il modello può essere riaddestrato quando arrivano dati nuovi.

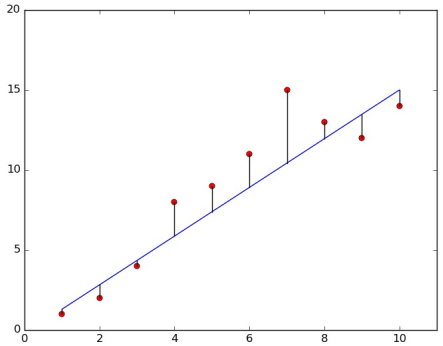
Il Machine Learning come problema di ottimizzazione

1. Si sceglie un **modello**, che dipende da un insieme di parametri $\boldsymbol{\theta}$.
2. Si sceglie una **metrica di errore** (*loss*) per valutare quanto il modello sbaglia.
3. Si **addestra** il modello: si regolano i parametri $\boldsymbol{\theta}$ per minimizzare l'errore sul *training set*.

Esempio: un problema di regressione

Abbiamo dei punti sul piano e vogliamo trovare la retta che li approssima meglio.

1. *Modello*: le funzioni lineari $y = ax + b$; i parametri sono a e b.
2. *Errore*: l'errore quadratico medio (MSE) tra i punti e la retta.
3. *Training*: si cercano i valori di a e b che rendono minimo l'MSE.



Perché allora si parla di *apprendimento* e non semplicemente di ottimizzazione? Per due motivi: i parametri vengono ricavati dall'**esperienza** (i dati di training), e quasi mai esiste una soluzione in forma chiusa, per cui si procede per piccoli passi successivi (ad esempio con la discesa del gradiente), in modo simile a un processo di apprendimento.
Lo scopo finale è fare **predizioni su dati nuovi**: assegnare una classe a un nuovo input (classificazione) o stimarne un valore numerico (regressione). I modelli servono anche a estrarre conoscenza dai dati, ad esempio trovando gruppi (*cluster*) o correlazioni.

Tipi di apprendimento

- **Supervisionato**: i dati hanno input e output (etichette). Esempi: classificazione, regressione.
- **Non supervisionato**: solo input, senza etichette. Esempi: clustering, analisi delle componenti (es. PCA), autoencoder.
- **Per rinforzo**: l'agente compie azioni e riceve ricompense; l'obiettivo è imparare a massimizzare il guadagno a lungo termine, tipicamente con una sorta di pianificazione *model-free* (senza un modello esplicito dell'ambiente).

Molte tecniche diverse

Non esiste un solo modo di fare machine learning: cambia sia il **modello** sia la **loss** con cui lo si valuta.

- **Modelli** (come si definisce la funzione): alberi di decisione, modelli lineari, reti neurali, ...
- **Funzioni di errore** (loss): errore quadratico medio (MSE), logistic loss, cross entropy, distanza coseno, massimo margine (*maximum margin*), ...

Scegliere una tecnica vuol dire quindi scegliere una coppia modello + loss adatta al problema.

1.1.2 Le feature e il Deep Learning

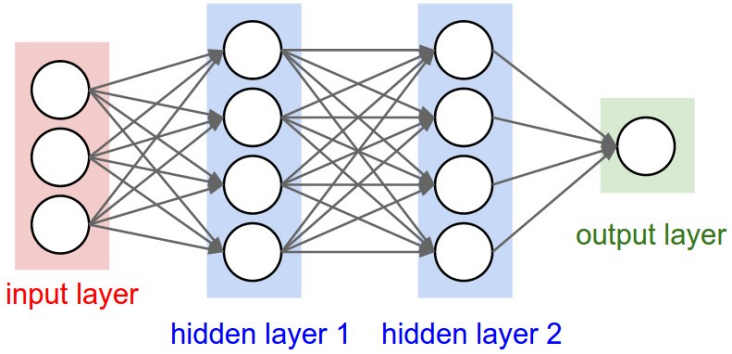
Una **feature** è una qualsiasi proprietà misurabile di un dato, utile a descriverlo. Le feature sono l'input del modello e la loro qualità influenza moltissimo il risultato. Sceglierle bene richiede però **conoscenza del dominio**.

Diagnosi medica	**Profilazione utente**	**Meteo**
Sintomi	Dati anagrafici	Temperatura
Condizioni generali	Interessi	Umidità
Storia clinica	Community	Pressione
Esami e referti	Stile di vita	Vento, pioggia, ...

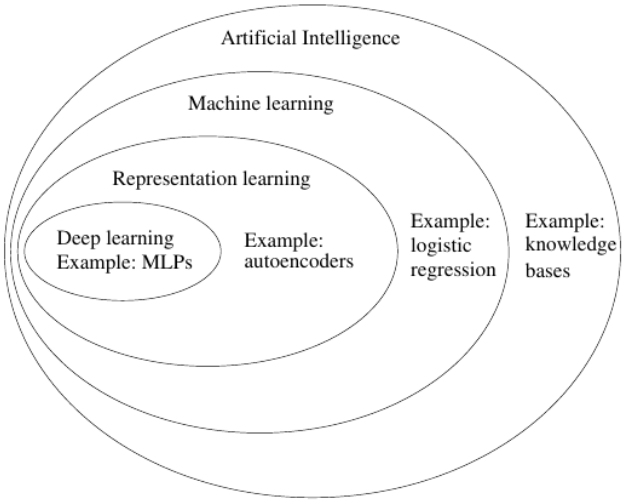
Il modo in cui si ottengono le feature distingue i vari approcci all'AI. Da sinistra, l'**esperto** ha sempre meno lavoro:

- **Sistemi a regole** (*knowledge-based*): si interroga un esperto su come risolve il problema e si cerca di imitare il suo ragionamento scrivendo a mano regole logiche. Nessun apprendimento. Input → Programma scritto a mano → Output

- **ML classico**: all'esperto si chiede solo quali siano le feature rilevanti per il problema (*feature engineering*); è la macchina a imparare la funzione che dalle feature porta all'output (es. regressione logistica, alberi di decisione).
  Input → Feature scelte a mano → Mappatura appresa → Output
- **Representation learning**: la macchina impara anche le feature, non solo la mappatura finale (es. autoencoder).
  Input → Feature apprese → Mappatura appresa → Output
- **Deep learning**: l'esperto sparisce del tutto: si danno alla rete i dati grezzi e si lascia a lei il compito di costruire da sola buone rappresentazioni interne. Una rete neurale è *profonda* se ha più strati nascosti: ogni strato calcola nuove feature, sempre più astratte, a partire da quelle dello strato precedente (es. MLP, reti percettive multilinea).
  Input → Feature semplici → Feature complesse → Mappatura → Output



Le discipline sono una dentro l'altra: AI ⊃ ML ⊃ Representation Learning ⊃ Deep Learning. Esempi per livello: le basi di conoscenza (*knowledge bases*) stanno nell'AI «classica», la regressione logistica nel ML, gli autoencoder nel representation learning, gli MLP nel deep learning.



Fonte: *Deep Learning*, Goodfellow, Bengio, Courville.

1.2 Alberi di decisione

1.2.1 Apprendimento, struttura ed espressività

Supponiamo che esista una funzione sconosciuta $f : X \to Y$ che associa a ogni input l'output corretto. Non conosciamo f, ma abbiamo un **training set** di esempi:

$$\mathcal{D} = \{\langle \mathbf{x}^{(1)}, y^{(1)} \rangle, \ldots, \langle \mathbf{x}^{(N)}, y^{(N)} \rangle\}$$

dove $\mathbf{x}^{(i)}$ è un vettore di feature e $y^{(i)}$ la sua etichetta. L'obiettivo è trovare una funzione che approssimi bene f sui dati noti e che **generalizzi** su quelli nuovi.

- Se Y è **discreto** (es. Sì/No, cifre 0–9) si parla di **classificazione**.
- Se Y è **continuo** (es. un prezzo, una temperatura) si parla di **regressione**.

Scegliere una tecnica di ML significa fissare una famiglia di funzioni candidate, lo **spazio delle ipotesi** $H = \{h \mid h : X \to Y\}$. Il **modello** è il modo in cui si calcola una specifica $h \in H$; addestrarlo vuol dire cercare in H la h che sbaglia di meno sul training set.

Input e output del task di apprendimento

- **Input**: un insieme di esempi di training $\{\langle \mathbf{x}^{(i)}, y^{(i)} \rangle\}$ relativi alla funzione obiettivo f.
- **Output**: l'ipotesi $h \in H$ che approssima meglio f.
- **Ma f è sconosciuta . . .** Non potendo confrontarci con f, di fatto cerchiamo la h che approssima meglio il training set, sperando che la cosa valga anche per dati nuovi.

Struttura di un albero di decisione

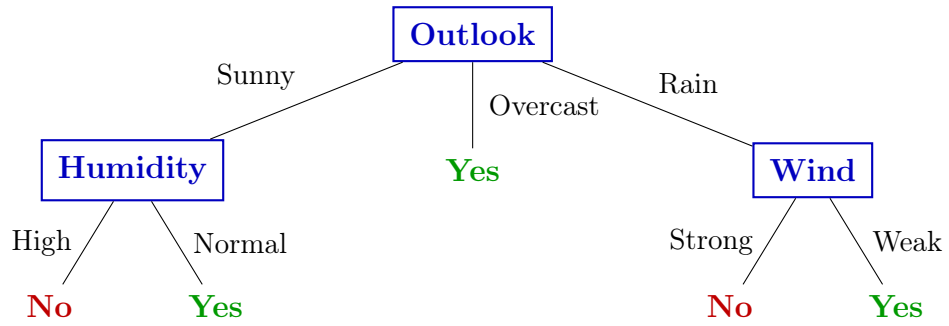
Un albero di decisione classifica un dato tramite una sequenza di domande:

- ogni **nodo interno** testa un attributo X_j (es. «Com'è il tempo?»);
- ogni **ramo** corrisponde a un valore dell'attributo (es. *Sunny*, *Overcast*, *Rain*);
- ogni **foglia** restituisce una classe y (o una distribuzione $P(Y \mid X)$).

Nel caso di *Play Tennis* la funzione obiettivo è

$$f : \text{Outlook} \times \text{Humidity} \times \text{Wind} \times \text{Temp} \to \{\text{Yes}, \text{No}\}.$$

Lo spazio delle ipotesi è il più libero possibile, $H = \{h \mid h : X \to Y\}$ senza altre restrizioni, e proviamo a rappresentare ogni $h \in H$ con un albero. Ogni esempio $\mathbf{x}$, poi, definisce un **cammino** nell'albero che termina in una foglia etichettata con la sua classe y.
Esempio classico, *Play Tennis*: ogni giornata è descritta da $\mathbf{x} = \langle \text{Outlook}, \text{Temperature}, \text{Humidity}, \text{Wind} \rangle$ e vogliamo prevedere se si gioca.



Outlook	Temperature	Humidity	Wind	Play Tennis?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Esempio di classificazione

Nuova giornata: $\langle \text{Sunny}, \text{Cool}, \text{High}, \text{Strong} \rangle$. Si parte dalla radice: Outlook = Sunny $\rightarrow$ si va su Humidity; Humidity = High $\rightarrow$ foglia **No**. Temperature e Wind non vengono nemmeno guardati: l'albero usa solo le feature che servono lungo quel cammino.

Espressività

Con attributi booleani $X_1, \ldots, X_n$, un albero può rappresentare qualsiasi funzione logica:

- **AND**, es. $Y = X_2 \wedge X_5$: si testa X_2; se è falso si risponde subito Falso, altrimenti si testa X_5.
- **OR**, es. $Y = X_4 \vee X_1$: se X_4 è vero si risponde Vero, altrimenti si testa X_1.
- formule più complesse, es. $Y = (X_2 \wedge X_5) \vee (\neg X_3 \wedge X_4 \wedge X_1)$.

Ogni cammino radice-foglia positiva è un AND di condizioni; l'albero intero è l'OR di questi cammini. Quindi un albero equivale a una formula in **forma normale disgiuntiva (DNF)**, e su attributi discreti può esprimere **qualunque** funzione booleana.
Da qui tre domande:

1. *Ogni $h \in H$ ha un albero che la rappresenta?* Sì: al limite basta un ramo per ogni riga della tabella di verità.
2. *L'albero è unico?* No: cambiando l'ordine dei test si ottengono alberi diversi equivalenti.

3. *Quale preferire?* Per il **rasoio di Occam**, il più semplice (meno nodi, meno profondo). Un albero enorme può non sbagliare mai sul training set ma imparare a memoria il rumore (*overfitting*); uno piccolo cattura solo le regolarità importanti e generalizza meglio.

1.2.2 Costruzione top-down, entropia e attributi continui

Trovare l'albero più piccolo in assoluto è NP-completo. Si usa quindi un algoritmo **greedy top-down** (ID3): si parte dalla radice e a ogni passo si sceglie l'attributo che sembra migliore *in quel momento*, senza tornare indietro.

1. Scegli l'attributo «migliore» per il nodo corrente.
2. Crea un figlio per ogni suo valore e distribuisci gli esempi tra i figli.
3. Per ogni figlio: se tutti gli esempi hanno la stessa classe, è una foglia; altrimenti ripeti dal punto 1.

Resta da capire cosa vuol dire «migliore». La risposta viene dalla teoria dell'informazione.

Informazione ed entropia

Un evento certo non ci dice nulla di nuovo; un evento raro ci sorprende molto. Si definisce l'**informazione** (o sorpresa) di un evento di probabilità p come

$$I(p) = -\log_2 p$$

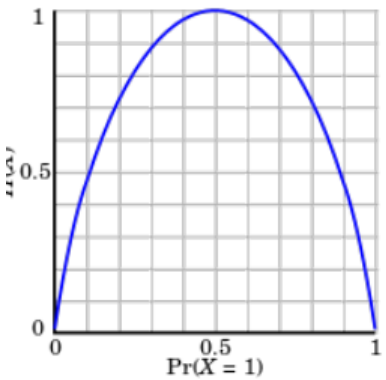
Il logaritmo è l'unica scelta che rende additiva l'informazione di eventi indipendenti: $I(p_1 p_2) = I(p_1) + I(p_2)$. L'unità di misura è il **bit**.
L'**entropia** di una variabile X con n valori possibili è la sorpresa media:

$$H(X) = -\sum_{i=1}^{n} P(X = i) \log_2 P(X = i)$$

(il segno meno serve perché $\log_2 p \leq 0$ quando $p \leq 1$). L'entropia misura quanto un insieme di dati è **impuro**:

- $H = 0$ se tutti gli esempi sono della stessa classe (nodo puro);
- $H = \log_2 n$ se le classi sono distribuite in modo uniforme (massima incertezza).

Nel caso binario la curva vale 1 bit al 50% e scende a 0 agli estremi:



L'entropia è anche il **numero medio minimo di bit** per codificare i messaggi di una sorgente. Con 8 simboli equiprobabili servono $\log_2 8 = 3$ bit; se alcuni simboli sono più frequenti, si danno loro codici più corti (come nella codifica di Huffman) e la lunghezza media scende fino a $H(X)$.

Information Gain

Dividendo un nodo con l'attributo X, vorremmo figli più puri del padre. L'incertezza residua è l'**entropia condizionata**, media delle entropie dei figli pesata per la loro dimensione:

$$H(Y \mid X) = \sum_v P(X = v)\, H(Y \mid X = v)$$

L'**information gain** è quanto l'entropia si riduce:

$$I(Y, X) = H(Y) - H(Y \mid X)$$

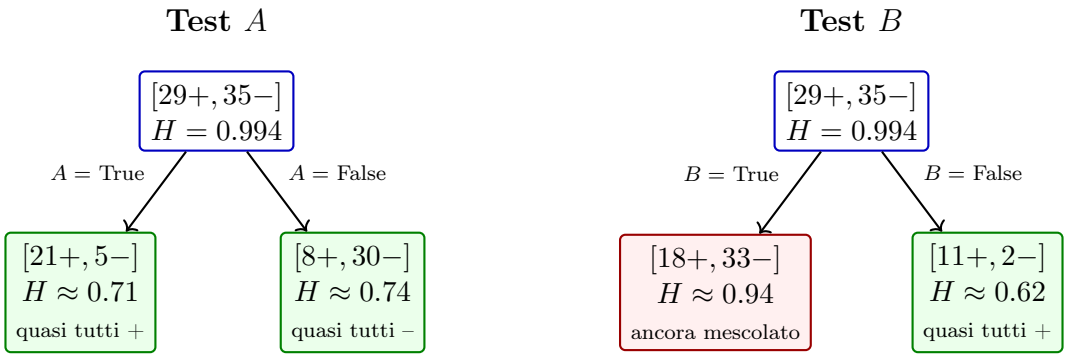
A ogni nodo si sceglie l'attributo con information gain **massimo**.

Esempio: meglio A o B?

Idea semplice: vogliamo il test che «mette ordine» meglio, cioè che separa i positivi (+) dai negativi (−). Un test è buono se, dopo averlo fatto, i due gruppi che ne escono sono **più omogenei** (più puri) di quello di partenza.
Siamo in un nodo con 64 esempi, $[29+, 35-]$: quasi 50 e 50, quindi molto confuso. L'entropia di partenza è alta:

$$H(Y) = -\tfrac{29}{64}\log_2 \tfrac{29}{64} - \tfrac{35}{64}\log_2 \tfrac{35}{64} \approx 0.994$$

Abbiamo due attributi booleani candidati, A e B. Vediamo come dividerebbero gli stessi 64 esempi:



Cosa si vede a occhio:

- **Con** A entrambi i figli sono abbastanza ordinati: a sinistra dominano i +, a destra i −. Il test ha davvero separato le due classi.
- **Con** B il figlio grande (51 esempi su 64) è ancora quasi 50/50, quindi il test ha fatto poco. Il figlio «bello» $[11+, 2-]$ è piccolo e conta poco.

Ora i numeri, un passo alla volta. Notazione: $[p+, n-] = p$ esempi positivi e n negativi; l'entropia di un nodo si calcola **solo** da questi due numeri, con la formula $H = -f_+ \log_2 f_+ - f_- \log_2 f_-$ dove $f_+ = \frac{p}{p+n}$ e $f_- = \frac{n}{p+n}$ sono le frazioni.

Passo 1: entropia di ogni nodo

Si applica la formula a ciascun nodo dell'albero (padre e figli). Esempio per il figlio A = True, $[21+, 5-]$, che ha 26 esempi:

$$H(Y \mid A = T) = -\tfrac{21}{26}\log_2 \tfrac{21}{26} - \tfrac{5}{26}\log_2 \tfrac{5}{26} \approx 0.706$$

Allo stesso modo $H(Y \mid A = F)$ per $[8+, 30-]$ (38 esempi) e così via per i nodi di B.

Passo 2: entropia dopo il test (media pesata)

Dopo il test A gli esempi si dividono in due gruppi, e l'incertezza che resta è la media delle due entropie, **pesata** per quanti esempi ha ciascun gruppo: $\frac{26}{64}$ vanno a sinistra, $\frac{38}{64}$ a destra (i pesi sommano a 1).

$$H(Y \mid A) = \tfrac{26}{64} \cdot \underbrace{0.706}_{H(Y|A=T)} + \tfrac{38}{64} \cdot \underbrace{0.742}_{H(Y|A=F)} \approx 0.727$$

Per B i pesi sono $\frac{51}{64}$ e $\frac{13}{64}$ (i 51 esempi di $[18+, 33-]$ e i 13 di $[11+, 2-]$):

$$H(Y \mid B) = \tfrac{51}{64} \cdot 0.936 + \tfrac{13}{64} \cdot 0.619 \approx 0.872$$

Passo 3: information gain

Gain = incertezza prima − incertezza dopo:

$$I(Y, A) = H(Y) - H(Y \mid A) = 0.994 - 0.727 = \mathbf{0.267} \qquad I(Y, B) = 0.994 - 0.872 = 0.122$$

(Con più cifre decimali il valore è 0.266; $0.994 - 0.726 = 0.268$ risente dell'arrotondamento.)

	Figlio True	**Figlio False**	
Test A	$[21+, 5-]$: $H = 0.706$, peso $\frac{26}{64}$	$[8+, 30-]$: $H = 0.742$, peso $\frac{38}{64}$	$H(Y \mid A) = 0.727$, **gain 0.267**
Test B	$[18+, 33-]$: $H = 0.936$, peso $\frac{51}{64}$	$[11+, 2-]$: $H = 0.619$, peso $\frac{13}{64}$	$H(Y \mid B) = 0.872$, gain 0.122

Da dove vengono i valori: i numeri dentro le parentesi quadre ($[29+, 35-]$, $[21+, 5-]$, ...) sono **dati** dell'esercizio (conteggi sul training set); tutto il resto (frazioni, entropie, pesi, gain) si calcola da lì. Si sceglie A: toglie più del doppio di incertezza. Il peso conta: il ramo B = False è pulito ($H = 0.619$), ma ha solo 13 esempi su 64, quindi non compensa l'altro ramo ancora confuso ($H = 0.936$ su 51 esempi).

Attributi continui

Se un attributo è continuo (temperatura, umidità, ...) non ha senso creare un ramo per ogni valore: ogni ramo avrebbe un solo esempio e l'albero imparerebbe a memoria i dati. Si usa invece una **soglia** Θ e un test binario $A < \Theta$.
La soglia si sceglie confrontando l'information gain di più candidati:

- punti a intervalli regolari tra minimo e massimo (veloce ma approssimato);
- oppure si ordinano gli esempi per quell'attributo e si provano i **punti medi tra valori consecutivi**; basta considerare quelli in cui la classe cambia.

Esempio

Umidità (%)	65	70	80	85
Play Tennis	Yes	Yes	No	No

La classe cambia solo tra 70 e 80, quindi l'unica soglia candidata è $\Theta = 75$. Il test Umidità < 75 separa perfettamente le due classi.

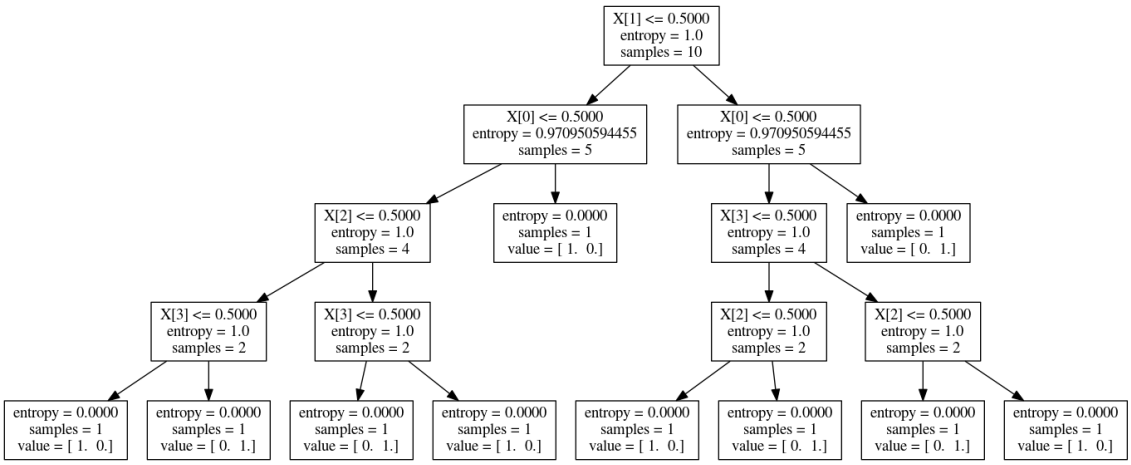
Limite della scelta greedy: lo XOR

L'algoritmo valuta **una feature alla volta**. Se la classe dipende da una combinazione di feature, può sbagliare. Esempio con `scikit-learn`, dove $Y = 1$ se e solo se $X_2 \neq X_3$:

```
from sklearn import tree
X = [[0,0,0,0], [1,0,0,0], [0,0,0,1], [0,0,1,0], [0,0,1,1],
     [0,1,0,0], [0,1,0,1], [1,1,0,1], [0,1,1,0], [0,1,1,1]]
Y = [0, 0, 1, 1, 0, 0, 1, 1, 1, 0]    # Y = X[2] xor X[3]

clf = tree.DecisionTreeClassifier(criterion='entropy', random_state=0)
clf = clf.fit(X, Y)
print(clf.predict([[1, 1, 1, 1]]))    # [1]
print(clf.feature_importances_)       # [0.17, 0.029, 0.4, 0.4]
```

Preso da solo, X_2 non dice nulla su Y: sapendo che $X_2 = 1$, Y vale 0 o 1 a seconda di X_3. Lo stesso vale per X_3. Il loro information gain alla radice è quasi nullo, quindi l'algoritmo sceglie per prima una feature irrilevante e l'albero diventa più profondo del necessario:



La **feature importance**, però, si calcola *a posteriori*: per ogni feature si sommano le riduzioni di impurità di tutti i nodi in cui viene usata, pesate per la frazione di esempi che passano da quel nodo, e poi si normalizza:

$$\text{Importance}(X_j) = \sum_{t \in \text{split}(X_j)} \frac{N_t}{N} \, \Delta\text{Impurità}(t)$$

Nell'esempio X_2 e X_3 ottengono il 40% ciascuna: a albero finito sono proprio loro a risolvere il problema.

1.2.3 Overfitting, potatura e impurità di Gini

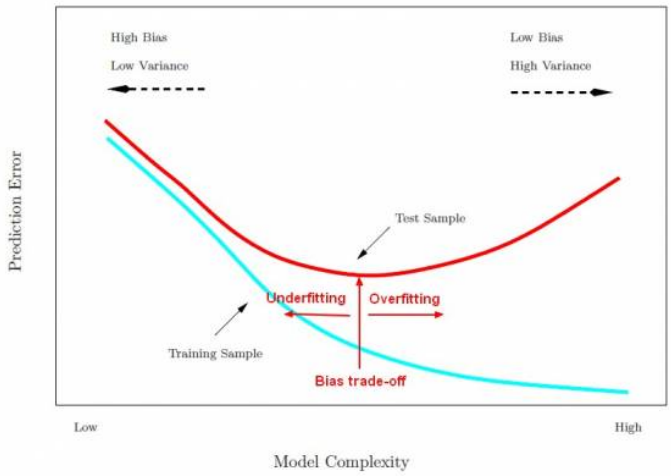
Se l'albero cresce finché ogni foglia è pura, sul training set avrà accuratezza del 100%, ma rischia di imparare a memoria anche il rumore. Questo è l'**overfitting**.
Formalmente, h va in overfitting se esiste un'altra ipotesi h' tale che

$$error_{train}(h) < error_{train}(h') \quad \text{ma} \quad error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$$

dove $error_{\mathcal{D}}$ è l'errore sull'intera distribuzione dei dati: h sembra migliore sui dati noti, ma è peggiore nella realtà.
L'errore dipende dalla **complessità** del modello:

- modello troppo semplice → **alto bias**, bassa varianza: sbaglia anche sul training set (**underfitting**);
- modello troppo complesso → basso bias, **alta varianza**: segue ogni oscillazione dei dati, e con un training set leggermente diverso darebbe un modello molto diverso (**overfitting**).

Al crescere della complessità l'errore sul training scende sempre, mentre quello sui dati nuovi ha un andamento a U. Cerchiamo il punto di minimo.



Poiché $error_{\mathcal{D}}$ non è calcolabile, si dividono i dati in due parti disgiunte: il **training set**, usato per costruire l'albero, e il **validation set**, usato solo dopo per stimare l'errore su dati mai visti.

Come evitare l'overfitting

- **Early stopping**: si smette di dividere un nodo se il gain è troppo basso, se ha pochi esempi o se si è raggiunta una profondità massima. Problema: è una scelta miope, e può fermarsi prima di split che sarebbero utili solo più in basso (come nello XOR).
- **Post-pruning**: si costruisce l'albero completo e poi lo si pota. In genere funziona meglio.

Reduced-error post-pruning

1. Costruisci l'albero completo sul training set.
2. Per ogni nodo interno, prova a sostituire il suo sottoalbero con una foglia etichettata con la classe più frequente, e misura l'accuratezza sul validation set.

3. Esegui la potatura che migliora di più l'accuratezza.

4. Ripeti finché nessuna potatura migliora l'accuratezza.

Esempio: un sottoalbero con tre foglie da 2 esempi l'una serve solo a gestire un'eccezione rumorosa. Con il sottoalbero l'accuratezza sul validation set è del 78%; sostituendolo con una foglia *Yes* sale all'84%. La potatura viene accettata.

Impurità di Gini

Al posto dell'entropia si può usare l'**impurità di Gini** (default in `scikit-learn`). Si pesca a caso un esempio dal nodo e gli si assegna una classe a caso, rispettando le proporzioni del nodo: Gini è la **probabilità di sbagliare**. Con f_i frazione di esempi della classe i:

$$I_G = \sum_{i=1}^{m} f_i(1 - f_i) = 1 - \sum_{i=1}^{m} f_i^2$$

Vale 0 per un nodo puro e 0.5 nel caso binario bilanciato. Di solito porta agli stessi split dell'entropia, ma è più veloce perché non usa logaritmi.

Esempio: di nuovo A o B

	Ramo True	**Ramo False**	**Media pesata**
A	$[21+, 5-]$: 0.310	$[8+, 30-]$: 0.332	**0.323**
B	$[18+, 33-]$: 0.456	$[11+, 2-]$: 0.260	0.416

Ad esempio $I_G(A = \text{True}) = 1 - (21/26)^2 - (5/26)^2 \approx 0.310$. L'impurità residua più bassa è quella di A: stessa scelta fatta con l'entropia.

1.2.4 Pregi, difetti, Random Forest e riepilogo

Pregi

- **Interpretabili**: ogni predizione si spiega con la sequenza di test lungo il cammino.
- Gestiscono feature numeriche e categoriche senza bisogno di normalizzarle.
- Predizione velocissima: costo proporzionale alla profondità, e l'altezza dell'albero è al più il numero di feature.

Difetti

- **Instabili**: piccole variazioni dei dati possono cambiare tutto l'albero (alta varianza).
- Tendono all'overfitting.
- Soffrono con classi sbilanciate: conviene bilanciare il dataset prima.

Random Forest

Si addestrano molti alberi *diversi* e si fa votare il gruppo: la combinazione di modelli poco correlati è più affidabile del singolo modello. Gli alberi vengono resi diversi in due modi:

- **Bagging**: ogni albero è addestrato su un campione estratto con reinserimento dal training set (contiene in media circa il 63% degli esempi originali).
- **Feature casuali**: a ogni split si considera solo un sottoinsieme casuale di feature (tipicamente $\sqrt{d}$ su d), così le feature dominanti non finiscono sempre alla radice.

La predizione finale è il **voto di maggioranza** (classificazione) o la **media** (regressione). Il risultato ha varianza molto più bassa di un singolo albero.

Uso con scikit-learn

Tutte le tecniche viste nella prima parte del corso si possono provare in poche righe con `scikit-learn`:

```
from sklearn import tree
X = [[0,0], [1,1]]
Y = [0, 1]
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, Y)
print(clf.predict([[0,1]]))   # [0]
```

Conviente sperimentare in prima persona con questi strumenti, senza però essere troppo naivi nel progetto d'esame.

Riepilogo

- Il problema: dato un training set $\{\langle \mathbf{x}^{(i)}, y^{(i)} \rangle\}$, cercare la miglior h nello spazio delle ipotesi H.
- L'apprendimento è una **ricerca/ottimizzazione** in H. Gli obiettivi possibili sono diversi: minimizzare l'errore sul training set (loss 0-1), oppure tra le soluzioni che sbagliano uguale preferire la più semplice (rasoio di Occam), ...
- Non basta minimizzare l'errore sul training: bisogna **generalizzare** su dati mai visti.
- **No Free Lunch**: nessun algoritmo è migliore di tutti gli altri su ogni problema. Senza un **bias induttivo** (assunzioni su ciò che ci aspettiamo dai dati) non è possibile imparare affatto; un algoritmo funziona bene solo se le sue assunzioni si adattano al problema.

1.3 Il paradigma probabilistico

1.3.1 Basi di probabilità

Finora abbiamo cercato una funzione $f : X \to Y$ che restituisce una sola risposta. I dati reali però sono rumorosi e ambigui. L'approccio probabilistico stima invece la distribuzione

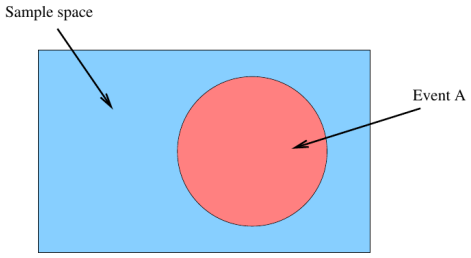
$$P(Y \mid X)$$

cioè quanto è probabile ciascuna classe dato l'input. Così il modello dice anche **quanto è sicuro** (una diagnosi al 99% non è come una al 51%) e gestisce casi in cui lo stesso input ha esiti diversi.

Concetti di base

- **Spazio campionario** Ω: l'insieme di tutti i possibili esiti di un esperimento. Esempio: tutti gli studenti presenti in un'aula.
- **Variabile aleatoria**: una funzione $X : \Omega \to$ Valori che assegna a ogni esito un valore; serve a misurare un esito del quale siamo incerti. Può essere *discreta* (es. «lo studente estratto a caso è maschio?» $\to$ True/False; il primo nome; «due studenti estratti a caso hanno lo stesso compleanno?») o *continua* (altezza: $\Omega \to \mathbb{R}$).
- **Evento**: un sottoinsieme $A \subseteq \Omega$, es. $A = \{x \in \Omega \mid \text{altezza}(x) \leq 175\}$.

$P(X)$ (la probabilità di X) è la **frazione di volte** in cui X si verifica, ripetendo lo stesso esperimento molte volte. Graficamente, per un evento A, è il rapporto tra l'area di A e l'area dell'intero Ω:



Ci interessano sia le probabilità di eventi singoli ($P(\text{gender} = m)$) sia quelle **condizionate** ad altri eventi ($P(\text{gender} = m \mid \text{height} \leq 175)$).

Attenzione a come si sceglie Ω

Il ragionamento funziona se tutti gli esiti di Ω sono **equiprobabili**. Lanciando due dadi, non conviene prendere $\Omega = \{2, \ldots, 12\}$ (la somma): ottenere 2 richiede solo $(1, 1)$, mentre 7 si ottiene in 6 modi. Meglio prendere le 36 coppie $\Omega = \{1..6\} \times \{1..6\}$, tutte con probabilità $1/36$, e definire la somma come variabile aleatoria S:

$$P(S = 7) = \tfrac{6}{36}, \qquad P(S = 2) = \tfrac{1}{36}$$

Assiomi

1. $0 \leq P(A) \leq 1$
2. $P(\text{True}) = 1$, $P(\text{False}) = 0$
3. $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

Da questi si ricavano due teoremi utili.

- **Complemento:** $P(\neg A) = 1 - P(A)$. Dimostrazione: applico il terzo assioma con $B = \neg A$:

  $$P(A \vee \neg A) = P(A) + P(\neg A) - P(A \wedge \neg A)$$

  ma $A \vee \neg A = \text{True}$ e $A \wedge \neg A = \text{False}$, quindi $1 = P(A) + P(\neg A) - 0$.
- **Partizione in due:** $P(A) = P(A \wedge B) + P(A \wedge \neg B)$. Dimostrazione: $A = A \wedge (B \vee \neg B) = (A \wedge B) \vee (A \wedge \neg B)$, e i due pezzi sono disgiunti, per cui le loro probabilità si sommano (l'intersezione è False, che vale 0).

Variabili con più valori

Se A assume uno tra k valori $\nu_1, \ldots, \nu_k$, questi sono mutuamente esclusivi ($P(A = \nu_i \wedge A = \nu_j) = 0$ per $i \neq j$) ed esaustivi ($P(A = \nu_1 \vee \cdots \vee A = \nu_k) = 1$, cioè $\sum_i P(A = \nu_i) = 1$). In generale, se $B_1, \ldots, B_k$ sono una partizione di Ω (sono disgiunti e la loro unione è Ω):

$$P(A) = \sum_{i=1}^{k} P(A \wedge B_i)$$

(**probabilità totale**): per calcolare la probabilità di A si può «spezzettare» A secondo ogni pezzo della partizione e sommare.

1.3.2 Probabilità condizionata e indipendenza

La probabilità di A **sapendo** che si è verificato B è

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$$

Intuitivamente: non considero più tutto Ω, ma solo la parte occupata da B, e mi chiedo quanta di quella parte è occupata da A. In termini di frequenze è esattamente il rapporto tra le volte in cui valgono insieme A e B e le volte in cui vale B.
Riordinando la definizione si ottiene la **chain rule** (regola del prodotto):

$$P(A \wedge B) = P(A \mid B)\, P(B) = P(B \mid A)\, P(A)$$

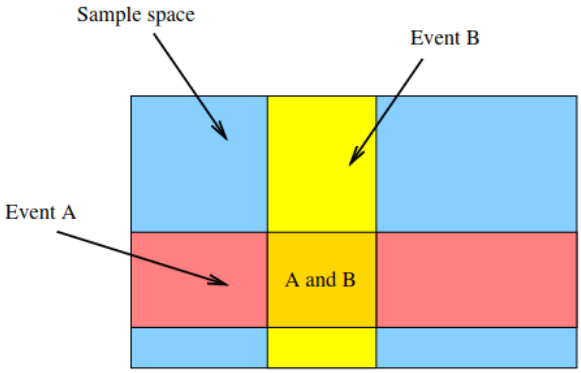
cioè la probabilità che valgano entrambi si calcola come la probabilità dell'uno per la probabilità dell'altro «visto» l'uno.
A e B sono **indipendenti** se sapere che B si è verificato non cambia la probabilità di A:

$$P(A \mid B) = P(A)$$

Equivalentemente, sostituendo nella chain rule:

$$P(A \wedge B) = P(A)\, P(B)$$

Dato che la chain rule vale anche scambiando i ruoli, vale anche $P(B \mid A) = P(B)$: l'indipendenza è **simmetrica**, nessuno dei due eventi influenza l'altro.
Graficamente, due eventi indipendenti sono «ortogonali»: B occupa la stessa frazione di A che occupa dell'intero Ω.



1.3.3 Teorema di Bayes

La chain rule scritta in due modi uguaglia lo stesso oggetto:

$$P(A \wedge B) = P(A \mid B)\, P(B) = P(B \mid A)\, P(A)$$

Dividendo entrambi i membri per $P(B)$ si ottiene il **teorema di Bayes**:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Bayes permette di «invertire» il condizionamento: conoscendo quanto spesso una causa A produce un effetto B, si ricava quanto è probabile la causa A dato che l'effetto B si è verificato.

Esempio: pantaloni o gonna?

In una scuola il 60% sono ragazzi e il 40% ragazze. Le ragazze portano pantaloni o gonna in egual misura, i ragazzi sempre pantaloni. Vediamo da lontano qualcuno con i pantaloni: qual è la probabilità che sia una ragazza?
Dati: $P(G) = 0.4$, $P(B) = 0.6$, $P(T \mid G) = 0.5$, $P(T \mid B) = 1$. Per la probabilità totale:

$$P(T) = P(T \mid G)P(G) + P(T \mid B)P(B) = 0.2 + 0.6 = 0.8$$

Quindi

$$P(G \mid T) = \frac{P(T \mid G)\, P(G)}{P(T)} = \frac{0.2}{0.8} = 0.25$$

Vedere i pantaloni abbassa la probabilità che sia una ragazza dal 40% al 25%.

Bayes per la classificazione

Vogliamo la classe $Y = y_i$ dato l'input $X = x_j$. La formula di Bayes dice

$$P(Y = y_i \mid X = x_j) = \frac{P(Y = y_i)\, P(X = x_j \mid Y = y_i)}{P(X = x_j)}$$

Il denominatore $P(X = x_j)$ si può espandere con la probabilità totale, sommando su tutte le classi:

$$P(X = x_j) = \sum_i P(X = x_j, Y = y_i) = \sum_i P(Y = y_i)\, P(X = x_j \mid Y = y_i)$$

e quindi

$$P(Y = y_i \mid X = x_j) = \frac{P(X = x_j \mid Y = y_i)\, P(Y = y_i)}{\sum_k P(X = x_j \mid Y = y_k)\, P(Y = y_k)}$$

I quattro termini della formula di Bayes

$$\underbrace{P(Y \mid X)}_{\text{posterior}} = \frac{\overbrace{P(X \mid Y)}^{\text{likelihood}} \cdot \overbrace{P(Y)}^{\text{prior}}}{\underbrace{\sum_{Y'} P(X \mid Y')\, P(Y')}_{\text{marginal likelihood}}}$$

- **Posterior**: probabilità della classe **dopo** aver visto i dati. È ciò che vogliamo calcolare.
- **Likelihood**: quanto è probabile osservare X se la classe è Y.
- **Prior**: probabilità della classe **prima** di vedere i dati.
- **Marginal likelihood**: $P(X)$, ottenuta sommando su tutte le classi; serve a normalizzare, cioè a far sommare a 1 le posterior. Si dice «marginal» perché si è **marginalizzata** la Y, cioè si è sommata (in generale integrata) via.

1.3.4 La distribuzione congiunta

Un altro modo di fare inferenza è costruire la **distribuzione congiunta**: una tabella con tutte le combinazioni di valori delle variabili, e per ognuna la sua probabilità (stimata contando le frequenze nei dati).

Gender	Hours	Wealth	Prob.
F	< 40	poor	0.25
F	< 40	rich	0.03
F	≥ 40	poor	0.04
F	≥ 40	rich	0.01
M	< 40	poor	0.33
M	< 40	rich	0.10
M	≥ 40	poor	0.13
M	≥ 40	rich	0.11

Le probabilità sommano a 1. Con n variabili booleane la tabella ha 2^n righe, quindi $2^n - 1$ parametri liberi.

Inferenza

La distribuzione congiunta permette di calcolare la probabilità di **qualsiasi evento** esprimibile come combinazione logica delle feature: basta sommare le righe che lo soddisfano:

$$P(E) = \sum_{\text{riga} \in E} P(\text{riga})$$

e una probabilità condizionata è un rapporto tra due somme:

$$P(E_1 \mid E_2) = \frac{P(E_1 \wedge E_2)}{P(E_2)} = \frac{\sum_{\text{riga} \in E_1 \wedge E_2} P(\text{riga})}{\sum_{\text{riga} \in E_2} P(\text{riga})}$$

Esempio: $P(M \wedge \text{poor}) = 0.33 + 0.13 = 0.46$ e $P(\text{poor}) = 0.25 + 0.04 + 0.33 + 0.13 = 0.75$, quindi

$$P(M \mid \text{poor}) = \frac{0.46}{0.75} \approx 0.61$$

Altro esempio (una posterior ricavata dalla congiunta): $P(M) = 0.46$ e $P(M \wedge \text{rich}) = 0.10 + 0.11 = 0.21$, quindi

$$P(\text{rich} \mid M) = \frac{0.21}{0.46} \approx 0.46$$

Fine della storia?

In teoria, con la congiunta $P(X_1, \ldots, X_n, Y)$ possiamo calcolare qualsiasi $P(Y \mid X)$, e quindi risolvere qualsiasi problema di classificazione. In pratica no:

- i parametri crescono come 2^n: con 30 feature booleane sono già più di un miliardo;
- per stimarli servirebbero quantità enormi di dati; con dati realistici quasi tutte le righe resterebbero vuote.

Servono quindi ipotesi semplificative sulla struttura dei dati: è l'idea alla base del **Naïve Bayes**. Esempio: la tabella di $P(\text{Wealth} \mid \text{Gender}, \text{Hours})$ ha già $4 = 2^2$ parametri da stimare.

Gender	**Hours**	$P(\text{rich} \mid X_1, X_2)$	$P(\text{poor} \mid X_1, X_2)$
F	≤ 40	0.09	0.91
F	> 40	0.21	0.79
M	≤ 40	0.23	0.77
M	> 40	0.38	0.62

Con n feature booleane i parametri diventano 2^n, e sono probabilità: per stimarle in modo affidabile serve un'enorme quantità di dati.

1.4 Naïve Bayes

Idea in una riga

Per classificare, confronta per ogni classe: **quanto è frequente la classe** $\times$ **quanto sono tipiche le feature osservate per quella classe**. La classe col punteggio più alto vince. Il trucco «naïve» è stimare ogni feature *separatamente*.

1.4.1 Ipotesi di indipendenza, classificazione ed esempio

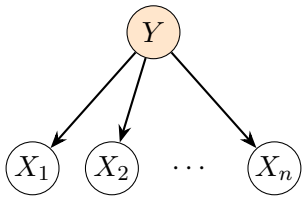
Vogliamo $P(Y \mid X_1, \ldots, X_n)$. Con Bayes basta stimare due cose:

$$P(Y = y_i \mid \mathbf{x}) = \frac{P(Y = y_i)\, P(\mathbf{x} \mid Y = y_i)}{\sum_k P(Y = y_k)\, P(\mathbf{x} \mid Y = y_k)}$$

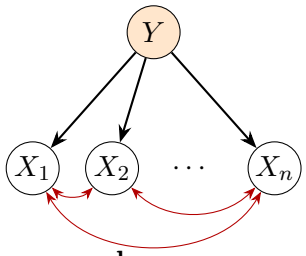
- $P(Y)$: con Y booleana serve **1** parametro;
- $P(X_1, \ldots, X_n \mid Y)$: una tabella congiunta per ogni valore di Y, cioè $2(2^n - 1)$ parametri. Il problema resta **esponenziale**.

Il **Naïve Bayes** fa un'ipotesi forte: **data la classe Y, le feature sono indipendenti tra loro**:

$$P(X_1, \ldots, X_n \mid Y) = \prod_{i=1}^{n} P(X_i \mid Y)$$



Naïve: le X_i dipendono solo da Y



Caso generale: servono anche le dipendenze tra le X_i

Indipendenza condizionata

X_i e X_j sono indipendenti dato Y se

$$P(X_i \mid X_j, Y) = P(X_i \mid Y)$$

cioè, una volta nota Y, conoscere X_j non aggiunge nulla su X_i. Due eventi possono essere dipendenti ma diventare indipendenti dato un terzo.

- *Esempio 1 (monete)*: una scatola contiene una moneta normale e una con due teste. Se ne sceglie una a caso e la si lancia due volte. A = «primo lancio testa», B = «secondo lancio testa», C = «moneta normale».
  - A e B **non** sono indipendenti: se il primo lancio dà testa, la moneta è probabilmente truccata, quindi anche il secondo darà testa. Infatti $P(A) = P(B) = \frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot 1 = \frac{3}{4}$ ma $P(A \wedge B) = \frac{1}{2} \cdot \frac{1}{4} + \frac{1}{2} \cdot 1 = \frac{5}{8} \neq \frac{9}{16} = P(A)P(B)$.
  - Dato C (sappiamo quale moneta è): i lanci sono indipendenti. Con la moneta normale $P(B \mid A, C) = \frac{1}{2} = P(B \mid C)$.
- *Esempio 2*: altezza e ricchezza di vocabolario non sono indipendenti (i bambini sono bassi e conoscono poche parole), ma lo diventano se conosciamo l'**età**: l'età è la causa comune che li lega.

Con due feature, per la chain rule e poi l'indipendenza condizionata:

$$P(X_1, X_2 \mid Y) = P(X_1 \mid X_2, Y)\, P(X_2 \mid Y) = P(X_1 \mid Y)\, P(X_2 \mid Y)$$

e in generale si ottiene il prodotto visto sopra.

Quanti parametri servono?

Con n feature booleane e Y booleana, per $P(X_1, \ldots, X_n \mid Y)$:

	formula	$n = 10$	$n = 30$
Senza ipotesi (congiunta)	$2(2^n - 1)$	2 046	$\approx 2{,}1 \cdot 10^9$
Naïve Bayes	$2n$	20	60

Per ogni feature e ogni classe basta $P(X_i = 1 \mid Y)$: si passa da **esponenziale a lineare**. Meno parametri significa anche che bastano pochi dati per stimarli bene.

Classificazione e training

Sostituendo l'ipotesi nella formula di Bayes:

$$P(Y = y_i \mid X_1, \ldots, X_n) = \frac{P(Y = y_i) \prod_j P(X_j \mid Y = y_i)}{P(X_1, \ldots, X_n)}$$

Il denominatore è uguale per tutte le classi: per classificare $\mathbf{x}^{new} = \langle x_1, \ldots, x_n \rangle$ basta confrontare i numeratori:

$$Y^{new} = \arg\max_{y_i} \underbrace{P(Y = y_i)}_{\text{prior}} \underbrace{\prod_j P(X_j = x_j \mid Y = y_i)}_{\text{likelihood}}$$

Algoritmo Naïve Bayes (variabili discrete)

Training: stimare

- per ogni classe y_k: $\pi_k = P(Y = y_k)$;
- per ogni feature X_i, ogni suo valore x_{ij} e ogni classe y_k: $\theta_{ijk} = P(X_i = x_{ij} \mid Y = y_k)$.

Le stime di massima verosimiglianza (**MLE**) sono semplici **conteggi** sul dataset $\mathcal{D}$:

$$\pi_k = \frac{\#\mathcal{D}\{Y = y_k\}}{|\mathcal{D}|} \qquad \theta_{ijk} = \frac{\#\mathcal{D}\{X_i = x_{ij} \wedge Y = y_k\}}{\#\mathcal{D}\{Y = y_k\}}$$

Classificazione di $\mathbf{a}^{new} = \langle a_1, \ldots, a_n \rangle$, dove $a_i = x_{ij}$:

$$Y^{new} = \arg\max_k \ \pi_k \prod_i \theta_{ijk}$$

In pratica: *training* = contare e dividere (nessuna ottimizzazione iterativa); *classificazione* = moltiplicare le probabilità lette dalle tabelle. Per questo Naïve Bayes è velocissimo.

Esempio: Play Tennis

Training set di 14 giornate (9 *Yes*, 5 *No*). Priors:

$$\pi_{Yes} = \tfrac{9}{14} \approx 0.64 \qquad \pi_{No} = \tfrac{5}{14} \approx 0.36$$

Per ogni classe contiamo quante volte compare ciascun valore, tra le sole giornate di quella classe (es. *Sunny* compare in 2 dei 9 *Yes*: $\theta_{Sunny,Yes} = 2/9$):

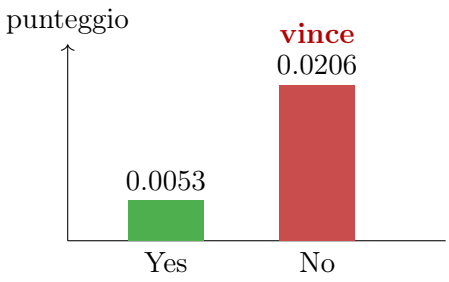
Valore	$\theta_{\cdot,Yes}$	$\theta_{\cdot,No}$	**Valore**	$\theta_{\cdot,Yes}$	$\theta_{\cdot,No}$
Sunny	2/9	3/5	High	3/9	4/5
Overcast	4/9	0	Normal	6/9	1/5
Rain	3/9	2/5	Weak	6/9	2/5
Hot	2/9	2/5	Strong	3/9	3/5
Mild	4/9	2/5			
Cool	3/9	1/5			

Controllo utile: per ogni feature e classe le probabilità sommano a 1 (es. Outlook per *Yes*: $\frac{2}{9} + \frac{4}{9} + \frac{3}{9} = 1$).

Predizione

Nuova giornata: $\langle \text{Sunny}, \text{Cool}, \text{High}, \text{Strong} \rangle$. Per ogni classe moltiplichiamo prior e quattro θ:

$$\text{Yes}: \ \tfrac{9}{14} \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \approx 0.0053 \qquad \text{No}: \ \tfrac{5}{14} \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} \cdot \tfrac{4}{5} \cdot \tfrac{3}{5} \approx 0.0206$$

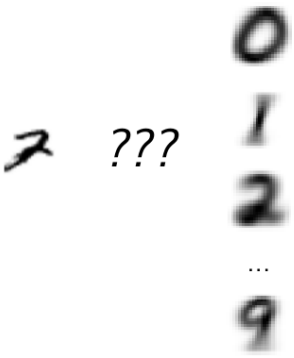


Vince **No**: non si gioca. I due numeri **non** sono probabilità (non sommano a 1). Per ottenerle si normalizza (si divide per la somma, che è $P(\mathbf{x})$):

$$P(No \mid \mathbf{x}) = \frac{0.0206}{0.0206 + 0.0053} \approx 0.80 \qquad P(Yes \mid \mathbf{x}) \approx 0.20$$

1.4.2 Natura generativa, limiti e stima MLE

Vogliamo $P(Y \mid X_1, \ldots, X_n)$, ma Bayes ci permette di **rovesciare il problema**: studiamo invece come sono distribuiti i dati *all'interno di ciascuna classe*, cioè $P(X_1, \ldots, X_n \mid Y = y_i)$. Per questo Naïve Bayes è un modello **generativo**: impara come «sono fatti» i dati di ogni classe, e potrebbe anche *generarne* di nuovi.
Esempio con MNIST (cifre scritte a mano): per ogni cifra $0, \ldots, 9$ si impara la distribuzione delle immagini di quella cifra. Un nuovo dato si classifica chiedendosi **a quale distribuzione è più probabile che appartenga**:



Le distribuzioni davvero interessanti sarebbero le congiunte $P(X_1, \ldots, X_n \mid Y = y_i)$: per MNIST, distribuzioni di immagini in uno spazio a 784 dimensioni (28×28 pixel), impossibili da stimare. L'ipotesi naïve tratta ogni pixel come indipendente dagli altri, riducendo il problema a stimare le singole $P(X_j \mid Y = y_i)$ per $j = 1, \ldots, n$: in pratica, per ogni cifra, «quanto spesso è acceso il pixel j».
Confronto che riprenderemo con la regressione logistica:

	Generativo (Naïve Bayes)	**Discriminativo** (reg. logistica)
Cosa stima	$P(X \mid Y)$ e $P(Y)$	direttamente $P(Y \mid X)$
Poi	applica Bayes	niente da invertire

Attenzioni e limiti

Il problema della probabilità zero

Le stime MLE di alcune probabilità possono valere **zero**. In Play Tennis, tra le giornate in cui *non* si gioca non compare mai *Overcast*:

$$\theta_{Overcast,No} = P(\text{Outlook} = \text{Overcast} \mid \text{Play} = \text{No}) = 0$$

Perché è un problema? La classificazione è un **prodotto**: un solo fattore nullo azzera tutto, qualunque cosa dicano gli altri. Con *Outlook* = *Overcast* il punteggio di *No* è sempre 0 e il modello risponde sempre *Yes*, anche se temperatura, umidità e vento gridano *No*. Ha imparato «Overcast → Yes» solo perché nel training non ha mai visto il contrario: un caso di mancanza di dati scambiata per impossibilità.
Rimedio: lo **smoothing** (addizione di Laplace). Si aggiunge $\alpha > 0$ (tipicamente $\alpha = 1$) ai conteggi, come se avessimo visto ogni valore almeno α volte:

$$\theta_{ijk} = \frac{\#\mathcal{D}\{X_i = x_{ij} \wedge Y = y_k\} + \alpha}{\#\mathcal{D}\{Y = y_k\} + \alpha \cdot |\text{valori}(X_i)|}$$

Nessun termine è più zero e i $\theta_{\cdot jk}$ sommano ancora a 1.

Esempio: Play Tennis con smoothing ($\alpha = 1$)

Outlook ha 3 valori, quindi $\theta_{Overcast,No} = \frac{0+1}{5+3} = \frac{1}{8}$ invece di 0. Applichiamo lo smoothing a tutte le feature e classifichiamo $\langle \text{Overcast}, \text{Hot}, \text{High}, \text{Strong} \rangle$ (Temp ha 3 valori, Humidity e Wind 2):

	Yes	**No**
Overcast	$\frac{4+1}{9+3} = \frac{5}{12}$	$\frac{0+1}{5+3} = \frac{1}{8}$
Hot	$\frac{2+1}{9+3} = \frac{3}{12}$	$\frac{2+1}{5+3} = \frac{3}{8}$
High	$\frac{3+1}{9+2} = \frac{4}{11}$	$\frac{4+1}{5+2} = \frac{5}{7}$
Strong	$\frac{3+1}{9+2} = \frac{4}{11}$	$\frac{3+1}{5+2} = \frac{4}{7}$
punteggio (con prior)	≈ 0.0089	≈ 0.0068

Senza smoothing *No* varrebbe esattamente 0. Ora le altre feature hanno voce: *Yes* vince ancora, ma di poco ($P(Yes \mid \mathbf{x}) \approx 0.56$), che è la risposta onesta per un caso davvero incerto.

E se l'indipendenza non vale?

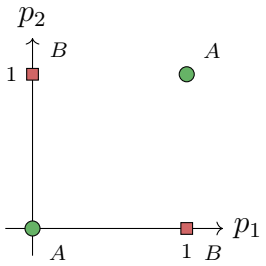
Quasi mai vale: i pixel vicini sono correlati, i sintomi di una malattia dipendono l'uno dall'altro. Le probabilità stimate sono quindi **sbagliate** in senso assoluto. Eppure il classificatore spesso funziona, perché:

- all'arg max non servono probabilità esatte: basta che vinca la classe giusta;
- gli errori dovuti alla dipendenza colpiscono tutte le classi in modo simile e si «smorzano» nel confronto.

Un effetto collaterale: Naïve Bayes tende a essere **troppo sicuro** (probabilità vicine a 0 o 1), perché conta più volte la stessa informazione (due feature correlate valgono come due prove indipendenti). Per decidere vale la sua classifica, meno il valore numerico delle probabilità.

Limite: quando è la relazione a fare la classe

Immagini casuali con 2 pixel $p_1, p_2 \in \{0, 1\}$. Classe A se $p_1 = p_2$, classe B altrimenti. I quattro possibili esempi:



Sotto A i pixel valgono insieme 0 o insieme 1, sotto B sono diversi, quindi per *ogni* pixel e *ogni* classe

$$P(p_1 = 1 \mid A) = P(p_2 = 1 \mid A) = P(p_1 = 1 \mid B) = P(p_2 = 1 \mid B) = 0.5$$

Per qualunque immagine Naïve Bayes confronta $\underbrace{0.5 \cdot 0.5}_{A} = \underbrace{0.5 \cdot 0.5}_{B}$: pareggio sempre. Ogni pixel, da solo, non dice nulla; l'informazione sta nella **relazione** tra i due, e un modello che li tratta come indipendenti la butta via. Collegamento con la geometria: i punti A e B stanno agli angoli opposti di un quadrato (problema **XOR**). Nessuna retta li separa, e Naïve Bayes booleano è un classificatore **lineare** (vedi sotto): coerente col fallimento.

Stima di massima verosimiglianza (MLE)

Finora abbiamo ricavato π_k e θ_{ijk} **contando** (frequenze relative). Perché proprio questi numeri? Perché sono la scelta che rende **massima la probabilità di aver osservato proprio i dati che abbiamo**: è la **stima di massima verosimiglianza** (*Maximum Likelihood Estimation*, MLE).

Esempio: una moneta truccata?

10 lanci: 6 Teste (T) e 4 Croci (C). Intuitivamente $P(T) = 0.6$. Supponiamo due sole ipotesi: moneta **onesta** ($\theta = 0.5$) o **truccata** ($\theta = 0.7$). Quale rende i dati più probabili?

Bernoulli e Binomiale

Ogni lancio è una variabile di **Bernoulli** con probabilità θ di testa. Il numero di teste in n lanci indipendenti segue la **binomiale**; con α_0 teste e $\alpha_1 = n - \alpha_0$ croci:

$$P(X_n = \alpha_0 \mid \theta) = \binom{n}{\alpha_0} \theta^{\alpha_0} (1-\theta)^{\alpha_1}$$

Il coefficiente $\binom{n}{\alpha_0} = \frac{n!}{\alpha_0!\,\alpha_1!}$ conta gli *ordini* possibili delle teste: non dipende da θ.

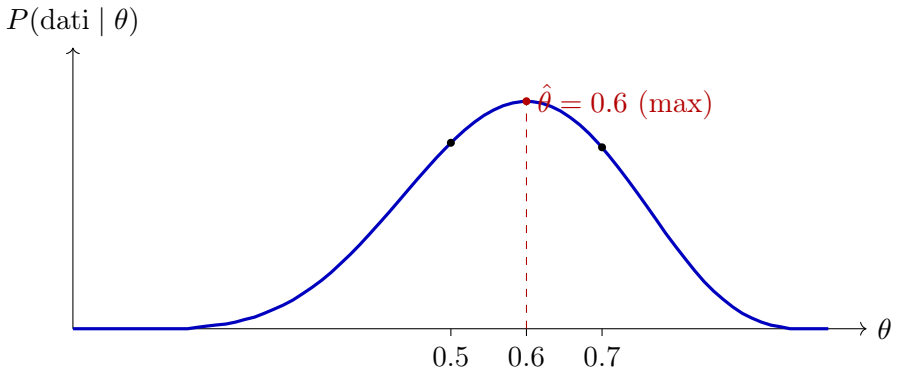
Confronto numerico

Con $n = 10$, $\alpha_0 = 6$, $\alpha_1 = 4$:

$$\theta = 0.5: \quad \binom{10}{6} (0.5)^{10} = 210 \cdot 0.00098 \approx \mathbf{0.205}$$

$$\theta = 0.7: \quad \binom{10}{6} (0.7)^6 (0.3)^4 = 210 \cdot 0.117649 \cdot 0.0081 \approx \mathbf{0.200}$$

Vince di poco $\theta = 0.5$. Ma tra *tutti* i $\theta \in [0, 1]$, qual è il migliore? Disegniamo la verosimiglianza in funzione di θ:



Il picco sta esattamente nella frequenza osservata, 6/10.

Calcolo formale della stima MLE

Cerchiamo $\hat{\theta} = \arg\max_\theta \binom{n}{\alpha_0} \theta^{\alpha_0} (1-\theta)^{\alpha_1}$. Il logaritmo è crescente: massimizzare la funzione equivale a massimizzare la **log-verosimiglianza** (il termine binomiale è costante in θ, lo

togliamo):

$$\ell(\theta) = \alpha_0 \ln \theta + \alpha_1 \ln(1-\theta)$$

Derivata uguale a zero:

$$\frac{\alpha_0}{\theta} - \frac{\alpha_1}{1-\theta} = 0 \iff \alpha_0(1-\theta) = \alpha_1\theta \iff \hat{\theta} = \frac{\alpha_0}{\alpha_0+\alpha_1} = \frac{\alpha_0}{n}$$

La stima MLE è la **frequenza empirica** osservata.

Caso con k valori: distribuzione multinomiale

Se la variabile ha k valori con probabilità $\theta_1, \ldots, \theta_k$ ($\sum_i \theta_i = 1$, come un dado a k facce), la probabilità di osservare α_i occorrenze del valore i in n prove è

$$P(X_n = \boldsymbol{\alpha} \mid \boldsymbol{\theta}) = c_{\boldsymbol{\alpha}} \prod_{i=1}^{k} \theta_i^{\alpha_i}, \qquad c_{\boldsymbol{\alpha}} = \frac{n!}{\alpha_1! \cdots \alpha_k!}$$

Massimizzando la log-verosimiglianza col vincolo $\sum_i \theta_i = 1$ (moltiplicatori di Lagrange) si ottiene ancora

$$\hat{\theta}_i = \frac{\alpha_i}{n}$$

Ecco perché π_k e θ_{ijk} di Naïve Bayes si ricavano con semplici conteggi: sono MLE di distribuzioni categoriali.

Legame con lo smoothing

MLE si fida ciecamente dei dati: con 0 occorrenze stima probabilità 0. Lo smoothing di Laplace corrisponde a partire da un'idea a priori («ogni valore è possibile») e correggerla con i dati. Se i dati sono tanti, le due stime quasi coincidono.

1.4.3 Testi, prodotto scalare e linearità di Naïve Bayes

Applicazione classica: **classificare testi** (spam/non spam, argomento di un articolo).

Rappresentazione del testo

Un documento è una sequenza di parole. Modelliamo la posizione i con una variabile X_i che assume valori nel vocabolario, e

$$\theta_{i,w,\ell} = P(X_i = w \mid Y = \ell)$$

è la probabilità che nella posizione i di un documento della classe ℓ compaia la parola w. Due assunzioni forti:

1. **Indipendenza** tra le posizioni (l'ipotesi naïve);
2. **Stazionarietà**: la probabilità di una parola non dipende da *dove* compare:

$$\theta_{i,w,\ell} = \theta_{j,w,\ell} = \theta_{w,\ell}$$

Il testo diventa un «sacchetto di parole» (*Bag of Words*): ordine e sintassi sono ignorati, conta solo *quante volte* compare ogni parola. «cane morde uomo» e «uomo morde cane» sono lo stesso documento.

Training e classificazione

- prior π_k: frazione di documenti del training set di classe y_k;
- $\theta_{w,k}$: frequenza di w rispetto a *tutte* le parole dei documenti di classe y_k (si concatenano i documenti della classe).

Per un nuovo documento $\mathbf{a}^{new} = \langle a_1, \ldots, a_n \rangle$:

$$Y^{new} = \arg\max_k \ \pi_k \prod_{i=1}^{n} \theta_{a_i,k}$$

Log-likelihood e raggruppamento delle parole

Moltiplicare centinaia di probabilità minuscole produce **underflow** (il numero diventa 0 per il calcolatore). Si usa il logaritmo, che trasforma prodotti in somme e non cambia l'arg max:

$$Y^{new} = \arg\max_k \left[\log \pi_k + \sum_{i=1}^{n} \log \theta_{a_i,k} \right]$$

Poiché θ non dipende dalla posizione, si raggruppano le parole uguali:

$$\sum_{i=1}^{n} \log \theta_{a_i,k} = \sum_{j \in \text{vocabolario}} n_j \log \theta_{j,k}$$

con n_j = numero di volte in cui la parola w_j compare nel documento.

Esempio: spam o no? (con smoothing $\alpha = 1$)

Training con 4 email, vocabolario di $|V| = 7$ parole.

Classe	Documenti
Spam	«offerta soldi ora», «soldi soldi gratis»
Ham	«ciao ora riunione», «riunione domani ciao»

Priors: $\pi_{spam} = \pi_{ham} = \frac{1}{2}$. Ogni classe ha 6 parole in totale, quindi $\theta_{w,k} = \frac{n_{w,k}+1}{6+7}$:

	offerta	soldi	ora	gratis	ciao	riunione	domani
$\theta_{\cdot,spam}$	$\frac{2}{13}$	$\frac{4}{13}$	$\frac{2}{13}$	$\frac{2}{13}$	$\frac{1}{13}$	$\frac{1}{13}$	$\frac{1}{13}$
$\theta_{\cdot,ham}$	$\frac{1}{13}$	$\frac{1}{13}$	$\frac{2}{13}$	$\frac{1}{13}$	$\frac{3}{13}$	$\frac{3}{13}$	$\frac{2}{13}$

Nuovo documento: «soldi gratis ora ora», cioè $n_{soldi} = 1$, $n_{gratis} = 1$, $n_{ora} = 2$.

$$\text{spam}: \ \tfrac{1}{2} \cdot \tfrac{4}{13} \cdot \tfrac{2}{13} \cdot \left(\tfrac{2}{13}\right)^2 \propto 4 \cdot 2 \cdot 4 = 32 \qquad \text{ham}: \ \tfrac{1}{2} \cdot \tfrac{1}{13} \cdot \tfrac{1}{13} \cdot \left(\tfrac{2}{13}\right)^2 \propto 1 \cdot 1 \cdot 4 = 4$$

Spam vince 8 a 1 (circa 89% in probabilità). Si noti che «ora» appare in entrambe le classi con lo stesso peso e non cambia il rapporto: contano le parole che *discriminano*.

Interpretazione come prodotto scalare

Immaginiamo uno spazio con una dimensione per ogni parola del vocabolario:

- per ogni classe k, un **vettore dei pesi** $\mathbf{s}_k = \langle \log \theta_{j,k} \rangle_{j \in V}$;
- per il documento, il **vettore dei conteggi** $\mathbf{d} = \langle n_j \rangle_{j \in V}$.

Tolta la prior, il punteggio è un prodotto scalare:

$$\sum_j n_j \log \theta_{j,k} = \mathbf{d} \cdot \mathbf{s}_k$$

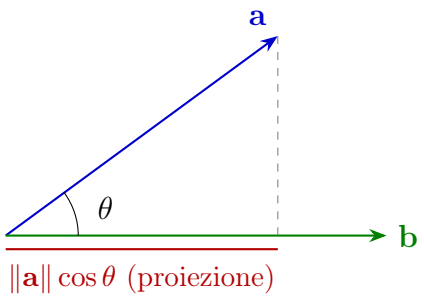
Classificare = scegliere la classe il cui profilo di parole è più «allineato» al documento.

Geometria del prodotto scalare e Cosine Similarity

Il prodotto scalare ha due definizioni equivalenti:

- **analitica**: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$ (si moltiplicano le componenti e si somma);
- **geometrica**: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos\theta$, con θ angolo tra i vettori e $\|\mathbf{a}\| = \sqrt{\sum_i a_i^2}$.

Interpretazione: il prodotto scalare misura quanto $\mathbf{a}$ «va nella direzione» di $\mathbf{b}$: è la lunghezza della proiezione di $\mathbf{a}$ su $\mathbf{b}$ ($\|\mathbf{a}\| \cos\theta$) per $\|\mathbf{b}\|$.



Tre casi da ricordare:

θ	$\cos\theta$	significato
$0°$ (stessa direzione)	1	massima somiglianza
$90°$ (ortogonali)	0	nessuna relazione (prodotto scalare nullo)
$180°$ (opposti)	-1	somiglianza opposta

Esempio numerico: $\mathbf{a} = (3, 4)$, $\mathbf{b} = (4, 0)$. Analitico: $3 \cdot 4 + 4 \cdot 0 = 12$. Geometrico: $\|\mathbf{a}\| = 5$, $\|\mathbf{b}\| = 4$, $\cos\theta = 12/20 = 0.6$ ($\theta \approx 53°$).

Dimostrazione dell'equivalenza

Sul triangolo di lati $\mathbf{a}$, $\mathbf{b}$, $\mathbf{a}-\mathbf{b}$, la legge dei coseni (Carnot) dà

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta \implies \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta = \frac{\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - \|\mathbf{a}-\mathbf{b}\|^2}{2}$$

In coordinate piane, con $\mathbf{a} = [a_1, a_2]$ e $\mathbf{b} = [b_1, b_2]$:

$$\frac{(a_1^2 + a_2^2) + (b_1^2 + b_2^2) - \big((a_1 - b_1)^2 + (a_2 - b_2)^2\big)}{2}$$
$$= \frac{a_1^2 + a_2^2 + b_1^2 + b_2^2 - a_1^2 + 2a_1b_1 - b_1^2 - a_2^2 + 2a_2b_2 - b_2^2}{2} = a_1b_1 + a_2b_2$$

Lo stesso vale in qualsiasi dimensione n.

Similarità coseno

Il prodotto scalare da solo premia i testi lunghi (vettori $\mathbf{d}$ con norma maggiore). Per confrontare solo la **direzione** si normalizza:

$$S_C(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \cos\theta \in [-1, 1]$$

Esempio: un tweet e un lungo saggio che parlano dello stesso argomento con le stesse frequenze relative di parole hanno vettori paralleli ($\cos\theta = 1$), anche se il saggio ha conteggi 100 volte più grandi. Per i conteggi di parole, che sono ≥ 0, il coseno sta in $[0, 1]$.

La natura lineare di Naïve Bayes (caso booleano)

Risultato importante: con feature booleane, **Naïve Bayes è un classificatore lineare**. La frontiera tra le classi è un iperpiano.

Punto di partenza: il rapporto tra le posteriori

Con $Y, X_i \in \{0, 1\}$, assegniamo $\mathbf{x}$ a $Y = 1$ se

$$\frac{P(Y = 1 \mid \mathbf{x})}{P(Y = 0 \mid \mathbf{x})} \geq 1$$

Con Bayes e indipendenza condizionata (il denominatore $P(\mathbf{x})$ si semplifica), poi il logaritmo:

$$\ln \frac{P(Y = 1)}{P(Y = 0)} + \sum_{i=1}^{n} \ln \frac{P(X_i = x_i \mid Y = 1)}{P(X_i = x_i \mid Y = 0)} \geq 0$$

Come far comparire x_i in modo lineare

Per $x \in \{0, 1\}$ vale $f(x) = x\, f(1) + (1 - x)\, f(0)$ (se $x = 1$ resta $f(1)$, se $x = 0$ resta $f(0)$). Poniamo

$$\theta_{i1} = P(X_i = 1 \mid Y = 1), \qquad \theta_{i0} = P(X_i = 1 \mid Y = 0)$$

da cui

$$\ln \frac{P(X_i = x_i \mid Y = 1)}{P(X_i = x_i \mid Y = 0)} = x_i \ln \frac{\theta_{i1}}{\theta_{i0}} + (1 - x_i) \ln \frac{1 - \theta_{i1}}{1 - \theta_{i0}}$$

Forma canonica di iperpiano

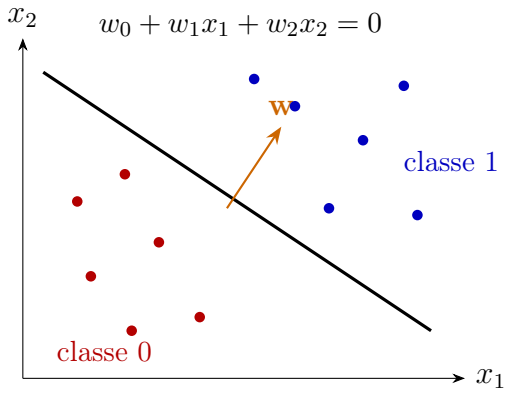
Raccogliamo i termini costanti e quelli che moltiplicano x_i:

$$\underbrace{\ln \frac{P(Y=1)}{P(Y=0)} + \sum_{i=1}^{n} \ln \frac{1-\theta_{i1}}{1-\theta_{i0}}}_{w_0} + \sum_{i=1}^{n} x_i \underbrace{\left[\ln \frac{\theta_{i1}}{\theta_{i0}} - \ln \frac{1-\theta_{i1}}{1-\theta_{i0}}\right]}_{w_i} \geq 0$$

Con $w_i = \ln \frac{\theta_{i1}(1-\theta_{i0})}{\theta_{i0}(1-\theta_{i1})}$ la regola diventa

$$w_0 + \sum_{i=1}^{n} w_i x_i \geq 0$$

La frontiera è $w_0 + \sum_i w_i x_i = 0$: un **iperpiano** (in 2D una retta).



Significato dei pesi

- Ogni feature contribuisce al punteggio con un peso **additivo** w_i, indipendente dalle altre: come un sistema di voti.
- $w_i > 0$ se $\theta_{i1} > \theta_{i0}$: la presenza di x_i spinge verso la classe 1; $w_i < 0$ nel caso opposto; $w_i = 0$ se la feature non distingue le classi.
- Esempio: se $\theta_{i1} = 0.8$ e $\theta_{i0} = 0.2$, allora $w_i = \ln \frac{0.8 \cdot 0.8}{0.2 \cdot 0.2} = \ln 16 \approx 2.77$: feature fortemente indicativa della classe 1.
- w_0 raccoglie la prior e i contributi delle feature quando valgono 0 (il «bias»).

Perché la linearità non è un limite

Un separatore lineare sembra rigido. Ma con tante feature (migliaia di parole, centinaia di pixel) i campioni sono molto **sparsi** nello spazio, e le classi risultano spesso separabili da un iperpiano: i classificatori lineari diventano sorprendentemente efficaci.

1.4.4 Attributi continui e valutazione del modello

Molti problemi di classificazione reali si basano su feature **continue** (ad esempio altezza, peso, temperatura, intensità dei pixel).

Il limite delle probabilità puntuali e densità di probabilità

Per una variabile aleatoria continua X, la probabilità che venga assunto uno specifico valore esatto è nulla:

$$P(X = x) = 0 \quad \forall x \in \mathbb{R}$$

Non possiamo quindi stimare $P(X_i = x \mid Y = y_k)$ tramite semplici conteggi di frequenza, poiché ogni valore reale può comparire una sola volta (o mai).
Per superare questo limite si introduce un **bias induttivo**: si assume che, condizionatamente a ciascuna classe y_k, la feature continua X_i segua una distribuzione nota descritta da una **funzione di densità di probabilità** (*Probability Density Function*, PDF), tipicamente la **distribuzione Gaussiana** (o Normale).

Ripasso: la distribuzione Gaussiana

Una variabile $X \sim \mathcal{N}(\mu, \sigma^2)$ ha funzione di densità:

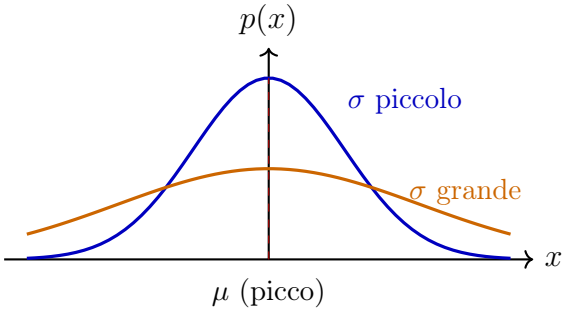
$$p(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Tale distribuzione gode di proprietà fondamentali:

- **Integrale unitario**: l'area totale sottesa dalla curva di densità è pari a 1:

$$\int_{-\infty}^{+\infty} p(x \mid \mu, \sigma)\, dx = 1$$

- **Media** $\mathbb{E}[X] = \mu$: individua il centro di simmetria e il **picco** (valore massimo) della distribuzione;
- **Varianza** $\mathbb{V}[X] = \sigma^2$ (deviazione standard σ): quantifica la dispersione. Più σ^2 è piccola, più la campana è ristretta e concentrata attorno a μ; viceversa, più σ^2 è grande, più la curva risulta appiattita e allargata;
- **Simmetria**: la funzione è perfettamente simmetrica rispetto al valore medio μ;
- **Massima entropia**: tra tutte le distribuzioni di probabilità continue definite su $\mathbb{R}$ aventi una data media e varianza, la Gaussiana è quella con **entropia maggiore**. Ciò la rende la scelta meno restrittiva e più naturale (secondo il principio di massima entropia).



Attenzione: $p(x)$ è una **densità**, non una probabilità: può assumere valori superiori a 1 se σ è sufficientemente piccolo. Il confronto tra classi rimane tuttavia rigoroso perché il fattore differenziale dx si elide nel rapporto di verosimiglianza.

Confronto tra classi ed esempio: peso medio cani e gatti

Supponiamo di voler distinguere due classi tramite una feature continua x (ad esempio il peso medio):

- **Classe 1 (Negativa, es. Gatti)**: $x \mid \text{Gatto} \sim \mathcal{N}(\mu_1, \sigma_1^2)$ (in rosso);
- **Classe 2 (Positiva, es. Cani)**: $x \mid \text{Cane} \sim \mathcal{N}(\mu_2, \sigma_2^2)$ (in blu), con $\mu_2 > \mu_1$ e $\sigma_1^2 < \sigma_2^2$.

Criterio del confronto geometrico: quando si osserva una nuova istanza con peso x, si traccia una retta verticale in corrispondenza del valore x sulle ascisse e si osservano i due punti di intersezione con le curve di densità:

$$p(x \mid \text{Gatto}) \quad \text{e} \quad p(x \mid \text{Cane})$$

Il punto di intersezione con ordinata maggiore (pesato per le rispettive probabilità a priori $P(Y)$) indicherà a quale distribuzione è più probabile che appartenga il valore osservato.

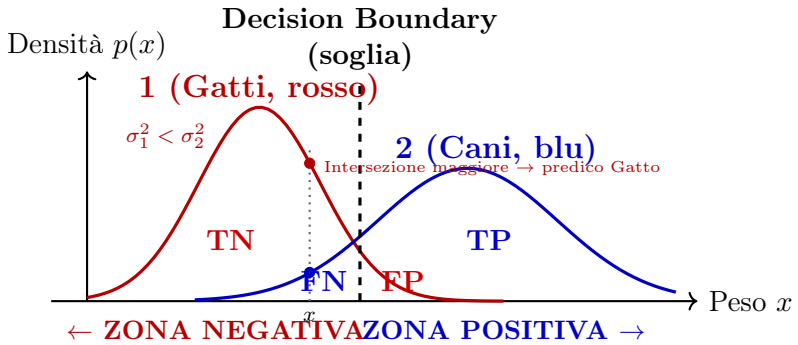
Decision Boundary (Frontiera di decisione)

La **frontiera di decisione** è la retta verticale che separa lo spazio delle feature in due zone distinte:

- **Zona negativa** (a sinistra della soglia): il modello classifica l'istanza come classe negativa (es. Gatto);
- **Zona positiva** (a destra della soglia): il modello classifica l'istanza come classe positiva (es. Cane).

La posizione della soglia può essere stabilita nel punto di intersezione naturale delle curve a posteriori (ottimo bayesiano per minimizzare l'errore globale), ma il progettista può **spostarla intenzionalmente** in base alle esigenze del problema:

- traslarla verso sinistra per **massimizzare il recall** (catturare più positivi reali ed evitare falsi negativi, tollerando più falsi allarmi);
- traslarla verso destra per **massimizzare la precisione** (ridurre i falsi positivi, a costo di perdere qualche positivo reale).



Formulazione generale e stima dei parametri

Con n feature continue $X_1, \ldots, X_n$:

1. Data la classe y_k, ciascuna feature X_i segue una distribuzione gaussiana indipendente:

$$P(X_i = x \mid Y = y_k) = \mathcal{N}(x \mid \mu_{ik}, \sigma_{ik}^2) = \frac{1}{\sigma_{ik}\sqrt{2\pi}} \exp\left(-\frac{(x-\mu_{ik})^2}{2\sigma_{ik}^2}\right)$$

2. Ciascuna classe ha prior $\pi_k = P(Y = y_k)$.

Classificazione

$$Y^{new} = \arg\max_k \ \pi_k \prod_{i=1}^{n} \mathcal{N}(a_i \mid \mu_{ik}, \sigma_{ik}^2)$$

Stime MLE dai dati

Per ogni classe si calcolano media e varianza campionarie sui soli esempi di quella classe (δ = funzione indicatrice):

$$\mu_{ik} = \frac{\sum_{j=1}^{N} x_i^{(j)} \delta(y^{(j)} = y_k)}{\sum_{j=1}^{N} \delta(y^{(j)} = y_k)} \qquad \sigma_{ik}^2 = \frac{\sum_{j=1}^{N} (x_i^{(j)} - \mu_{ik})^2 \, \delta(y^{(j)} = y_k)}{\sum_{j=1}^{N} \delta(y^{(j)} = y_k)}$$

Il modello richiede $2n$ parametri continui per classe (una media e una varianza per feature), mantenendo una complessità lineare $\mathcal{O}(n)$.

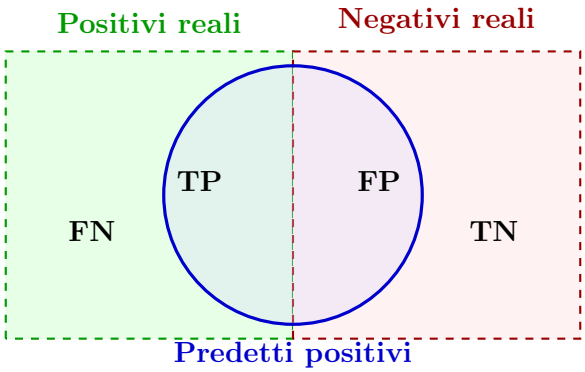
Valutazione del modello: metriche di errore e matrice di confusione

Poiché le predizioni del modello possono essere errate, introduciamo le metriche di confronto basate sulla suddivisione dei casi:

		Classe reale	
		Positivo (es. Cani)	**Negativo (es. Gatti)**
Predizione del modello	**Positivo**	**TP** (True Positive)	**FP** (False Positive)
	Negativo	**FN** (False Negative)	**TN** (True Negative)

Interpretazione geometrica e semantica:

- **True Negative (TN)**: negativi reali che cadono nella zona negativa (es. gatti classificati come gatti);
- **False Positive (FP)**: negativi reali la cui feature supera la soglia finendo nella zona positiva (es. un gatto particolarmente pesante scambiato per cane $\rightarrow$ falso allarme);
- **False Negative (FN)**: positivi reali che cadono a sinistra della soglia nella zona negativa (es. cani con peso inferiore alla soglia che vengono "persi" ed erroneamente etichettati come gatti);
- **True Positive (TP)**: positivi reali che cadono correttamente nella zona positiva (es. cani classificati come cani).



Metriche di prestazione

- **Accuratezza (Accuracy)**: quota di predizioni corrette sul totale dei casi osservati:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Totale}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

  Può risultare fuorviante in presenza di classi fortemente sbilanciate.

- **Precisione (Precision)**: misura la purezza e la qualità delle predizioni positive:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

  Alta precisione $\implies$ pochissimi falsi allarmi (FP minimi).

- **Recall (Richiamo o Sensibilità)**: misura la capacità del modello di intercettare i casi positivi reali:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

  Un richiamo alto indica che il modello individua la stragrande maggioranza dei positivi reali, riducendo al minimo i casi persi (FN minimi).

- **F1-score**: media armonica tra Precisione e Richiamo:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

  Penalizza i modelli sbilanciati in cui una delle due metriche è sacrificata a favore dell'altra.

Esempio numerico: filtro antispam

Su 100 email analizzate, 10 sono spam (positivi) e 90 ordinarie (negativi). Il modello ne segnala 9 come spam: ne individua 6 corrette e compie 3 falsi allarmi.

$$\text{TP} = 6, \quad \text{FP} = 3, \quad \text{FN} = 4, \quad \text{TN} = 87$$

$$\text{Accuracy} = \frac{6+87}{100} = 0.93 \qquad \text{Precision} = \frac{6}{6+3} \approx 0.67 \qquad \text{Recall} = \frac{6}{6+4} = 0.60$$

$$F_1 = 2 \cdot \frac{0.67 \cdot 0.60}{0.67 + 0.60} \approx 0.63$$

L'accuracy del 93% nasconde il fatto che un classificatore banale (che predice sempre "ordinaria") avrebbe un'accuracy del 90% con un recall pari a 0. Precision, Recall e F_1 evidenziano invece la reale affidabilità del classificatore.

Riepilogo: Naïve Bayes

- **Idea**: $\arg\max_k \ \pi_k \prod_i P(X_i \mid y_k)$, assumendo le feature condizionatamente indipendenti data la classe.
- **Training**: conteggi diretti (MLE) per feature discrete; stima di medie e varianze empiriche per feature continue (Gaussian Naïve Bayes).
- **Parametri**: scalabilità lineare $\mathcal{O}(n)$ anziché esponenziale $\mathcal{O}(2^n)$.
- **Cautele**: smoothing di Laplace per evitare probabilità nulle; passaggio ai logaritmi per contrastare l'underflow numerico.
- **Natura**: modello generativo; con feature booleane definisce una frontiera di decisione lineare (iperpiano).
- **Limite**: non modella le correlazioni tra feature (es. problema dello XOR) e tende a generare probabilità a posteriori sovrastimate.
- **Valutazione**: matrice di confusione, Accuracy, Precision, Recall e F_1-score.

1.5 Regressione Logistica

Idea in una riga

Calcola un punteggio lineare $z = w_0 + \sum_i w_i x_i$ e lo trasforma in probabilità con la **sigmoide**. I pesi **w** non si ricavano da conteggi (come in Naïve Bayes) ma si **apprendono per discesa del gradiente**, massimizzando la verosimiglianza dei dati.

BIG PICTURE: cosa stiamo facendo e perché

Il problema. Abbiamo una variabile da prevedere $Y \in \{0, 1\}$ (es. spam / non spam) e un vettore di feature $X = \langle x_1, \ldots, x_n \rangle$. Per decidere ci basta conoscere $P(Y \mid X)$: si sceglie la classe più probabile.

La strada di Naïve Bayes (indiretta). Si stimano $P(Y)$ e $P(X \mid Y)$ e poi si inverte con Bayes. Costo: bisogna modellare *come sono fatte le feature* ($P(X \mid Y)$ per ogni classe), e per farlo si fanno ipotesi forti (indipendenza condizionata) che spesso sono false.

L'idea della regressione logistica (diretta). Il nostro scopo è solo classificare, quindi perché non stimare *direttamente* $P(Y \mid X)$?. Ma stimare $P(Y \mid X)$ senza struttura è impossibile: avremmo un valore diverso per ogni possibile X. Serve quindi una **forma funzionale** con pochi parametri $w_0, w_1, \ldots, w_n$.

Il filo logico, in 5 passi:

1. **Quale forma scegliere?** Non la inventiamo: la *deduciamo* da Naïve Bayes gaussiano. Dimostriamo che, se $P(X_i \mid Y = k)$ sono gaussiane con stessa varianza nelle due classi, allora automaticamente $P(Y = 1 \mid X) = \dfrac{1}{1 + \exp(-w_0 - \sum_i w_i x_i)}$, cioè una **sigmoide di una combinazione lineare**.

2. **Cambio di prospettiva.** Una volta ottenuta la forma, la prendiamo *come ipotesi* e lasciamo i pesi w_i **liberi**, senza più legarli a medie e varianze. Così il modello funziona anche se le gaussiane non valgono.

3. **Come scegliere i pesi?** Massima verosimiglianza (MLE): i pesi migliori sono quelli che rendono più probabili le etichette y^ℓ osservate nel training set.

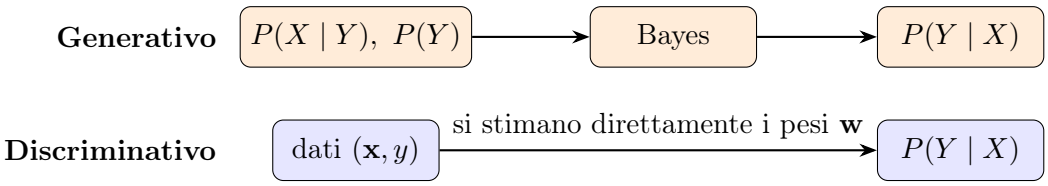
4. **Come calcolarli?** Il problema di ottimizzazione non ha soluzione in forma chiusa: si procede per **discesa del gradiente**, a piccoli passi. Calcoliamo il gradiente con precisione e scopriamo che è semplicissimo: $\sum_\ell x_i^\ell (y^\ell - \alpha^\ell)$.

5. **Come evitare l'overfitting?** Si aggiunge una penalità sui pesi (**regolarizzazione**).

Obiettivo finale: ottenere un classificatore con frontiera lineare, ma i cui pesi sono scelti per imparare *direttamente* a classificare bene, non a descrivere i dati.

1.5.1 Dall'approccio generativo a quello discriminativo e funzione logistica

Con Naïve Bayes abbiamo seguito un approccio **generativo**: per ottenere $P(Y \mid X)$ abbiamo prima modellato $P(Y)$ e $P(X \mid Y)$, poi usato Bayes.
Ma se lo scopo è classificare, **perché non stimare direttamente** $P(Y \mid X)$, senza spendere parametri e dati per modellare come sono distribuite le feature? È l'approccio **discriminativo**, di cui la **regressione logistica** è il modello fondamentale.



Passo 1 della big picture: da dove viene la forma di $P(Y \mid X)$

Dimostriamo che la forma logistica *non è arbitraria*: è la conseguenza esatta di un Naïve Bayes gaussiano.

Ipotesi

- Y variabile booleana con distribuzione di Bernoulli di parametro π: $P(Y = 1) = \pi$, $P(Y = 0) = 1 - \pi$;
- le X_i sono variabili continue, **condizionatamente indipendenti** data Y;
- $P(X_i \mid Y = k) = \mathcal{N}(X_i; \mu_{ik}, \sigma_i)$, cioè

$$P(X_i \mid Y = k) = \frac{1}{\sigma_i \sqrt{2\pi}}\, e^{-\frac{(X_i - \mu_{ik})^2}{2\sigma_i^2}}$$

  Attenzione: la deviazione standard σ_i dipende solo da i, **non dalla classe** k. La media μ_{ik} invece dipende da entrambe.

Tesi: $P(Y = 1 \mid X = \langle x_1, \ldots, x_n \rangle) = \dfrac{1}{1 + \exp\big(w_0 + \sum_i w_i x_i\big)}$ per opportuni w_0, w_i.

Dimostrazione. Seguiamo ogni passaggio.

(1) Teorema di Bayes. Il denominatore $P(X)$ si espande con la formula della probabilità totale sulle due classi:

$$P(Y = 1 \mid X) = \frac{P(Y = 1)\, P(X \mid Y = 1)}{P(Y = 1)\, P(X \mid Y = 1) + P(Y = 0)\, P(X \mid Y = 0)}$$

(2) Dividiamo numeratore e denominatore per il numeratore. Poniamo $A = P(Y = 1)P(X \mid Y = 1)$ e $B = P(Y = 0)P(X \mid Y = 0)$ (entrambi > 0). Allora $P(Y = 1 \mid X) = \frac{A}{A+B} = \frac{1}{1+B/A}$, cioè

$$P(Y = 1 \mid X) = \frac{1}{1 + \dfrac{P(Y = 0)\, P(X \mid Y = 0)}{P(Y = 1)\, P(X \mid Y = 1)}}$$

(3) Scriviamo il rapporto come esponenziale. Per ogni $a > 0$ vale $a = \exp(\ln a)$; con $a = B/A$:

$$P(Y = 1 \mid X) = \frac{1}{1 + \exp\Big(\ln \dfrac{P(Y = 0)\, P(X \mid Y = 0)}{P(Y = 1)\, P(X \mid Y = 1)}\Big)}$$

Perché lo facciamo: il logaritmo trasforma prodotti e rapporti in somme, e le gaussiane (che sono esponenziali) in polinomi.

(4) Scomponiamo il logaritmo. Usiamo $\ln \frac{ab}{cd} = \ln \frac{a}{c} + \ln \frac{b}{d}$, i valori $P(Y = 0) = 1 - \pi$, $P(Y = 1) = \pi$, e l'**indipendenza condizionata** $P(X \mid Y = k) = \prod_i P(X_i \mid Y = k)$:

$$\ln \frac{P(Y = 0)\, P(X \mid Y = 0)}{P(Y = 1)\, P(X \mid Y = 1)} = \ln \frac{1 - \pi}{\pi} + \ln \frac{\prod_i P(X_i \mid Y = 0)}{\prod_i P(X_i \mid Y = 1)} = \ln \frac{1 - \pi}{\pi} + \sum_i \ln \frac{P(X_i \mid Y = 0)}{P(X_i \mid Y = 1)}$$

(l'ultimo passo è $\ln \prod = \sum \ln$, applicato al rapporto dei due prodotti, che è il prodotto dei rapporti).

(5) Rapporto di gaussiane. Sostituiamo la densità. Il fattore $\frac{1}{\sigma_i \sqrt{2\pi}}$ è **identico** al numeratore e al denominatore, quindi si semplifica: *è qui che serve avere la stessa σ_i nelle due classi.* Resta

$$\ln \frac{P(X_i \mid Y = 0)}{P(X_i \mid Y = 1)} = \ln \frac{e^{-\frac{(X_i - \mu_{i0})^2}{2\sigma_i^2}}}{e^{-\frac{(X_i - \mu_{i1})^2}{2\sigma_i^2}}} = \frac{-(X_i - \mu_{i0})^2 + (X_i - \mu_{i1})^2}{2\sigma_i^2}$$

(si è usato $\ln(e^a / e^b) = a - b$).

(6) Sviluppiamo i quadrati. Con $(X_i - \mu_{ik})^2 = X_i^2 - 2\mu_{ik} X_i + \mu_{ik}^2$ il numeratore diventa

$$-X_i^2 + 2\mu_{i0} X_i - \mu_{i0}^2 + X_i^2 - 2\mu_{i1} X_i + \mu_{i1}^2 = 2(\mu_{i0} - \mu_{i1}) X_i + (\mu_{i1}^2 - \mu_{i0}^2)$$

I termini X_i^2 **si elidono** (ancora grazie alla stessa σ_i): il risultato è *lineare* in X_i. Dividendo per $2\sigma_i^2$:

$$\ln \frac{P(X_i \mid Y = 0)}{P(X_i \mid Y = 1)} = \frac{\mu_{i0} - \mu_{i1}}{\sigma_i^2} X_i + \frac{\mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}$$

(7) Sommiamo su i e raccogliamo. Dal passo (4):

$$\ln \frac{P(Y = 0)\, P(X \mid Y = 0)}{P(Y = 1)\, P(X \mid Y = 1)} = \underbrace{\ln \frac{1 - \pi}{\pi} + \sum_{i=1}^{n} \frac{\mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}}_{w_0} + \sum_{i=1}^{n} \underbrace{\frac{\mu_{i0} - \mu_{i1}}{\sigma_i^2}}_{w_i} X_i = w_0 + \sum_i w_i X_i$$

Inserendo nel passo (3) otteniamo la tesi:

Risultato

$$P(Y = 1 \mid X = \langle x_1, \ldots, x_n \rangle) = \frac{1}{1 + \exp\big(w_0 + \sum_i w_i x_i\big)}$$

con

$$w_0 = \ln \frac{1 - \pi}{\pi} + \sum_i \frac{\mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}, \qquad w_i = \frac{\mu_{i0} - \mu_{i1}}{\sigma_i^2}$$

■

Conseguenze

Poniamo $s = w_0 + \sum_i w_i x_i$. Dal risultato, $P(Y=0 \mid X) = 1 - P(Y=1 \mid X) = 1 - \frac{1}{1+e^s} = \frac{e^s}{1+e^s}$. Dividendo le due probabilità:

$$\frac{P(Y=0 \mid X)}{P(Y=1 \mid X)} = \frac{\frac{e^s}{1+e^s}}{\frac{1}{1+e^s}} = e^s \quad \Longrightarrow \quad \ln \frac{P(Y=0 \mid X)}{P(Y=1 \mid X)} = w_0 + \sum_i w_i x_i$$

Con questa convenzione di segno, si sceglie la classe 0 se $w_0 + \sum_i w_i x_i > 0$ (log-rapporto positivo: $Y=0$ è più probabile).

Cambio di segno e definizione della regressione logistica

La regressione logistica assume la forma con il segno meno:

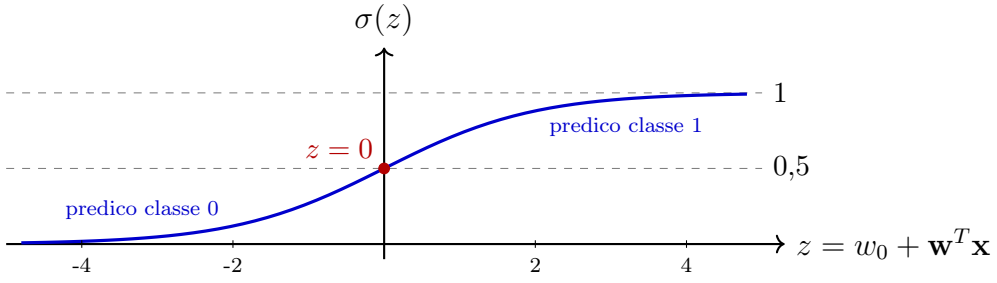
$$P(Y=1 \mid X = \langle x_1, \ldots, x_n \rangle) = \frac{1}{1 + \exp\big(-w_0 - \sum_i w_i x_i\big)}$$

Il cambio di segno equivale a sostituire ogni peso w con $-w$ (cioè $w_0 = \ln \frac{\pi}{1-\pi} - \sum_i \frac{\mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}$ e $w_i = \frac{\mu_{i1} - \mu_{i0}}{\sigma_i^2}$): è solo convenzione, ma ha il vantaggio che ora $w_0 + \sum_i w_i x_i > 0$ significa **classe 1**, e un peso positivo spinge verso la classe 1 (infatti $w_i > 0 \iff \mu_{i1} > \mu_{i0}$: se la media della feature è più alta nella classe 1, valori alti di x_i indicano la classe 1).

La funzione sigmoide (o logistica)

$$P(Y=1 \mid \mathbf{x}) = \frac{1}{1 + \exp\left(-\left(w_0 + \sum_{i=1}^n w_i x_i\right)\right)} = \sigma\left(w_0 + \sum_{i=1}^n w_i x_i\right), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Passo 2 della big picture. Sotto le ipotesi gaussiane di Naïve Bayes, $P(Y \mid X)$ *è* una sigmoide di una combinazione lineare. La regressione logistica **prende questa forma come ipotesi** e lascia i pesi $\mathbf{w}$ liberi (non più legati a μ_{ik}, σ_i, π): è per questo che funziona anche quando le ipotesi gaussiane non valgono, ed è più robusta.



Proprietà della sigmoide

- Trasforma qualsiasi $z \in (-\infty, +\infty)$ in un numero in $(0,1)$: interpretabile come probabilità.
- È strettamente crescente e simmetrica rispetto a $(0, 0.5)$: $\sigma(-z) = 1 - \sigma(z)$.

- Per $z \to +\infty$ tende a 1, per $z \to -\infty$ a 0.

z	-4	-2	-1	0	1	2	4
$\sigma(z)$	0.018	0.119	0.269	0.5	0.731	0.881	0.982

Odds e log-odds

Poiché $P(Y = 0 \mid \mathbf{x}) = 1 - P(Y = 1 \mid \mathbf{x})$:

$$\frac{P(Y=1 \mid \mathbf{x})}{P(Y=0 \mid \mathbf{x})} = \frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}} = e^{z} \qquad \Longrightarrow \qquad \ln \frac{P(Y=1 \mid \mathbf{x})}{P(Y=0 \mid \mathbf{x})} = z = w_0 + \sum_{i=1}^{n} w_i x_i$$

Il rapporto è detto *odds* e il suo logaritmo *log-odds* (o *logit*): **la regressione logistica assume che i log-odds siano lineari nelle feature**. Interpretazione: aumentare x_i di 1 somma w_i ai log-odds, cioè moltiplica gli odds per e^{w_i}.

Regola di decisione

$$P(Y=1 \mid \mathbf{x}) \geq P(Y=0 \mid \mathbf{x}) \iff z \geq 0 \iff w_0 + \sum_{i=1}^{n} w_i x_i \geq 0$$

La frontiera è l'iperpiano $w_0 + \sum_i w_i x_i = 0$. Allontanandosi dalla frontiera $|z|$ cresce e la probabilità si avvicina a 0 o 1: la sigmoide misura la *confidenza*.

Esempio: ore di studio e promozione

Una sola feature $x =$ ore di studio, pesi $w_0 = -4$, $w_1 = 1$, quindi $z = x - 4$.

x (ore)	0	2	4	6	8
$z = x - 4$	-4	-2	0	2	4
$P(\text{promosso} \mid x)$	0.02	0.12	0.50	0.88	0.98

La frontiera è $x = 4$ ore. Ogni ora in più moltiplica gli odds per $e^1 \approx 2.7$.

1.5.2 Addestramento e discesa del gradiente

Qui i pesi non si ottengono da medie e varianze: si **ottimizzano direttamente sul dataset**. Il ragionamento ha tre mosse: (1) misuriamo *quanto bene* un certo $\mathbf{w}$ spiega i dati (verosimiglianza); (2) trasformiamo la misura in una funzione da minimizzare (loss); (3) scendiamo lungo la loss a piccoli passi (gradiente). Prima fissiamo la notazione, perché in questa sezione compaiono molti simboli.

Notazione usata in questa sezione

Simbolo	Significato
n	numero di feature
N	numero di esempi nel training set
$\ell \in \{1, \dots, N\}$	**indice dell'esempio** (è un apice: x^ℓ è l'esempio ℓ-esimo, *non* una potenza)
$i \in \{0, \dots, n\}$	**indice della feature/del peso**
x_i^ℓ	valore della feature i nell'esempio ℓ
$y^\ell \in \{0, 1\}$	etichetta *vera* dell'esempio ℓ (dato nel training set)
$\mathbf{w} = (w_0, \dots, w_n)$	pesi del modello: sono le *incognite* da ottimizzare
$x_0^\ell = 1$	convenzione: feature fittizia sempre uguale a 1, così il bias w_0 rientra nella somma
$z^\ell = \sum_{i=0}^{n} w_i x_i^\ell$	punteggio lineare dell'esempio ℓ (con la convenzione sopra, $= w_0 + \sum_{i \geq 1} w_i x_i^\ell$)
$\alpha^\ell = \sigma(z^\ell)$	probabilità *prevista* dal modello che $Y = 1$ per l'esempio ℓ
$L(\mathbf{w})$	verosimiglianza: probabilità che il modello assegna a *tutte* le etichette del training set
$\mathrm{LL}(\mathbf{w}) = \ln L(\mathbf{w})$	log-verosimiglianza (da *massimizzare*)
$E(\mathbf{w}) = -\mathrm{LL}(\mathbf{w})$	loss / cross-entropy (da *minimizzare*)
$\eta > 0$	learning rate (lunghezza del passo)
$\nabla_{\mathbf{w}} E$	gradiente: vettore delle derivate parziali $\partial E / \partial w_i$

Attenzione: y^ℓ è un *dato* (0 o 1), α^ℓ è una *previsione* (un numero in $(0, 1)$ che dipende da $\mathbf{w}$). La loro differenza $y^\ell - \alpha^\ell$ è l'errore del modello sull'esempio ℓ.

La funzione di verosimiglianza

Per un singolo esempio $(\mathbf{x}^\ell, y^\ell)$ il modello dice: $P(Y = 1 \mid \mathbf{x}^\ell) = \alpha^\ell$ e $P(Y = 0 \mid \mathbf{x}^\ell) = 1 - \alpha^\ell$. Possiamo scrivere le due casistiche in un'unica formula (Bernoulli):

$$P(y^\ell \mid \mathbf{x}^\ell, \mathbf{w}) = (\alpha^\ell)^{y^\ell} (1 - \alpha^\ell)^{1 - y^\ell}$$

Se $y^\ell = 1$ l'esponente di $(1 - \alpha^\ell)$ è 0 e resta α^ℓ; se $y^\ell = 0$ resta $1 - \alpha^\ell$. In entrambi i casi è *la probabilità che il modello assegna all'etichetta corretta*: vogliamo che sia alta.

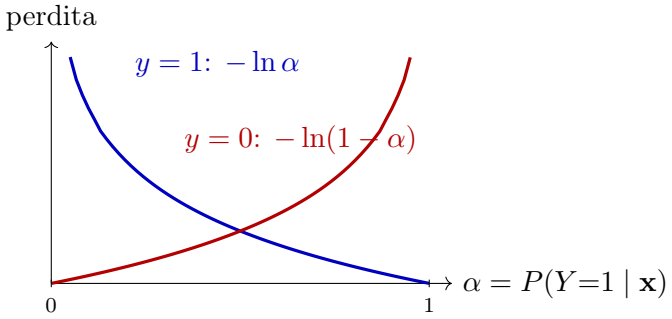
Assumendo gli esempi indipendenti, la probabilità di vedere *tutte* le etichette è il prodotto, e passando al logaritmo (che trasforma il prodotto in somma e non cambia il punto di massimo, perché ln è crescente):

$$L(\mathbf{w}) = \prod_{\ell=1}^{N} (\alpha^\ell)^{y^\ell} (1 - \alpha^\ell)^{1 - y^\ell} \qquad \mathrm{LL}(\mathbf{w}) = \sum_{\ell=1}^{N} \Big[y^\ell \ln \alpha^\ell + (1 - y^\ell) \ln(1 - \alpha^\ell) \Big]$$

Cerchiamo $\mathbf{w}^*$ che massimizza $\mathrm{LL}(\mathbf{w})$. Per convenzione l'ottimizzazione si formula come *minimizzazione*: si minimizza la **cross-entropy** $E(\mathbf{w}) = -\mathrm{LL}(\mathbf{w})$, che è la *loss* della regressione logistica. Massimizzare LL e minimizzare E sono la stessa cosa.

Intuizione sulla loss

Per un singolo esempio la perdita è $-\ln \alpha$ se $y = 1$ e $-\ln(1-\alpha)$ se $y = 0$: è piccola quando il modello è sicuro e ha ragione, **enorme** quando è sicuro e ha torto.



Assenza di soluzione analitica

Ponendo a zero le derivate di $\mathrm{LL}(\mathbf{w})$ si ottiene un sistema di equazioni non lineari **senza soluzione in forma chiusa** (a differenza della regressione lineare ai minimi quadrati). Serve un metodo numerico iterativo: la **discesa del gradiente**.

La tecnica del gradiente (Gradient Descent)

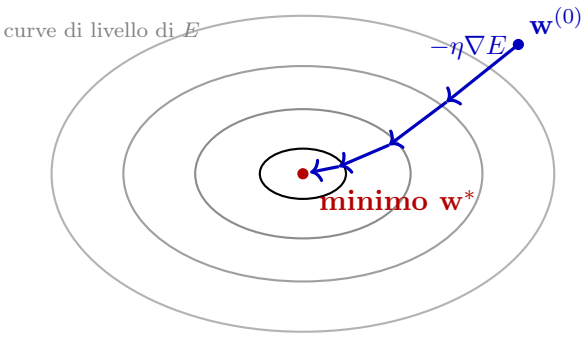
Idea. Ogni scelta di $\mathbf{w}$ ha un costo $E(\mathbf{w})$. Immaginiamo un paesaggio: sul piano orizzontale ci sono i pesi (un asse per ogni w_i), in verticale la loss $E(\mathbf{w})$. Cerchiamo il punto più basso ($\mathbf{w}^*$), ma non possiamo vedere tutto il paesaggio: in ogni punto sappiamo solo *da che parte è la discesa più ripida*, e facciamo un passo in quella direzione.
Il gradiente. Per ogni peso w_i la derivata parziale $\partial E/\partial w_i$ dice *di quanto cambia E se aumentiamo solo w_i di un pochino* (positiva: E sale, quindi conviene *diminuire* w_i; negativa: conviene aumentarlo). Impilando tutte le derivate parziali si ottiene il **gradiente**

$$\nabla_{\mathbf{w}} E = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right]^T$$

che punta nella direzione di **massima salita** di E. Per scendere ci si muove nella direzione **opposta**: ad ogni iterazione si *aggiorna* il vettore dei pesi

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} E \qquad \text{cioè, componente per componente:} \qquad w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}$$

dove la freccia $\leftarrow$ significa «sostituisci il vecchio valore con quello a destra» e $\eta > 0$ è il **learning rate** (lunghezza del passo). Poiché $E = -\mathrm{LL}$, si ha $\nabla E = -\nabla \mathrm{LL}$: lo stesso aggiornamento si scrive come *salita* sulla log-verosimiglianza, $w_i \leftarrow w_i + \eta\, \partial \mathrm{LL}/\partial w_i$. Si ripete finché i pesi smettono di cambiare in modo apprezzabile (gradiente ≈ 0).



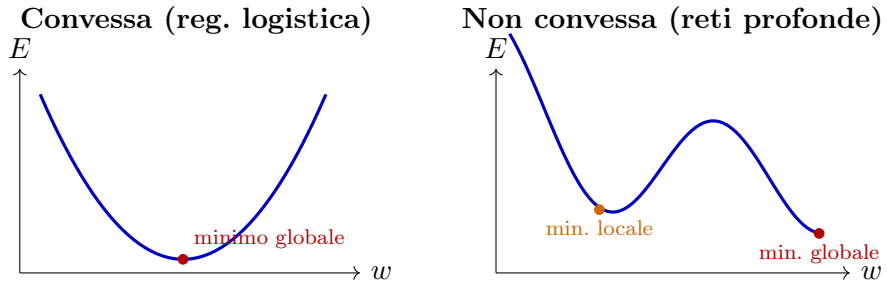
Effetto del learning rate

Vista in una sola dimensione ($E(w) = w^2$, derivata $2w$): con $w = 4$ e diversi η, dopo un passo $w \leftarrow w - \eta \cdot 2w$:

η	nuovo w	comportamento
0.05	3.6	troppo piccolo: converge, ma lentissimo
0.4	0.8	buono: si avvicina rapidamente al minimo
1.1	-4.8	troppo grande: supera il minimo e **diverge**

Convessità

La loss della regressione logistica è **convessa**: una «coppa» senza minimi locali, quindi il gradiente scende verso il minimo globale da qualunque punto iniziale (con η adeguato). Non è vero per le reti neurali, che hanno superfici molto più irregolari.



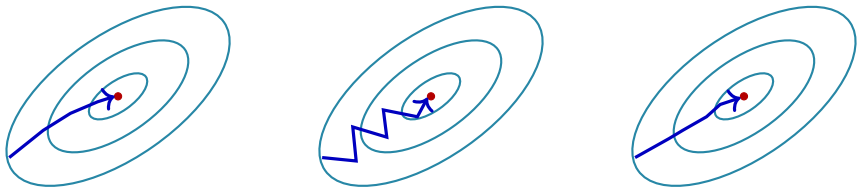
In una funzione convessa ogni segmento tra due punti del grafico sta sopra il grafico: ne segue che **ogni minimo locale è globale**. Nella superficie non convessa il gradiente si ferma nel primo avvallamento che incontra (dove $\nabla E = 0$), che può essere un minimo locale o un *punto di sella*, e il risultato dipende dai pesi iniziali. Per questo nelle reti profonde si parte da pesi casuali e si usano varianti più robuste del gradiente (momentum, Adam).

Quanti esempi per ogni passo?

Il gradiente di E è una somma sugli N esempi, quindi possiamo scegliere *quanti* esempi usare per stimarlo:

- **Batch**: gradiente esatto su tutti gli N esempi; direzione di massima discesa (perpendicolare alle curve di livello) ma costo $O(N)$ per ogni singolo passo: lento con molti dati.
- **Online / stocastico (SGD)**: un esempio alla volta; ogni passo costa poco ma il gradiente di un solo esempio è una stima *rumorosa* di quello vero, quindi la traiettoria fa zig-zag attorno alla discesa ideale (in media però va nella direzione giusta).
- **Mini-batch**: piccoli gruppi casuali (32–128 esempi); compromesso e standard nella pratica: meno rumore dell'online, e sfrutta bene il calcolo vettoriale su CPU/GPU.

Un'**epoca** è un passaggio completo sul training set: con N esempi e mini-batch di dimensione B un'epoca contiene N/B aggiornamenti. Se i dati sono ordinati per classe l'SGD si «sbilancia» verso l'ultima classe vista: si rimescola il dataset a ogni epoca.



Batch **Online (SGD)** **Mini-batch**

Calcolo esplicito del gradiente (dimostrazione completa)

Obiettivo. Per applicare l'aggiornamento serve $\dfrac{\partial \mathrm{LL}(\mathbf{w})}{\partial w_i}$, con $\mathrm{LL}(\mathbf{w}) = \sum_{\ell=1}^{N} \big[y^\ell \ln \alpha^\ell + (1 - y^\ell) \ln(1 - \alpha^\ell)\big]$.

Notazione. Valgono i simboli della tabella sopra: $z^\ell = \sum_{i=0}^{n} w_i x_i^\ell$ (con $x_0^\ell = 1$) e $\alpha^\ell = \sigma(z^\ell) = \dfrac{1}{1 + e^{-z^\ell}}$. Poiché la derivata di una somma è la somma delle derivate, calcoliamo prima il contributo di *un solo* esempio e poi sommiamo; per alleggerire si fissa l'esempio e si scrivono z, α, y, x_i omettendo l'apice ℓ .

Strategia. $\mathrm{LL}(\mathbf{w})$ dipende da w_i solo attraverso z, che a sua volta dipende da w_i. Quindi si usa la regola della catena: $\dfrac{\partial}{\partial w_i} = \dfrac{\partial}{\partial z} \cdot \dfrac{\partial z}{\partial w_i}$. Calcoliamo i pezzi uno alla volta.

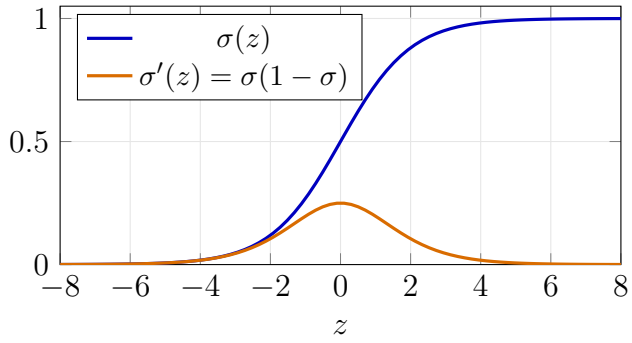
(a) Derivata della sigmoide. Scriviamo $\sigma(z) = (1 + e^{-z})^{-1}$ e deriviamo (regola della potenza e della catena):

$$\frac{\partial \sigma}{\partial z} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{?}$$

Il secondo fattore vale $\frac{e^{-z}}{1+e^{-z}} = \frac{(1+e^{-z})-1}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z)$. Dunque

$$\boxed{\frac{\partial \sigma}{\partial z} = \sigma(z)\big(1 - \sigma(z)\big)}$$

(La derivata della sigmoide è molto «piatta»: vale al massimo $\frac{1}{4}$ per $z = 0$ e tende a 0 per $|z|$ grande.)



Saturazione. Quando $|z|$ è grande la sigmoide è «satura» e $\sigma' \approx 0$. Sembrerebbe un problema (gradiente che svanisce), ma nella regressione logistica il fattore σ' *si cancella* con il $\frac{1}{\alpha}$ del logaritmo (passo (b)): il gradiente finale $x_i(y - \alpha)$ resta grande quando il modello è sicuro *e sbaglia* ($y - \alpha \approx$

± 1). Con una loss quadratica $(y-\alpha)^2$ il σ' non si cancellerebbe e l'apprendimento si bloccherebbe proprio sugli errori più gravi. Nelle reti profonde con molte sigmoidi il problema invece si presenta (*vanishing gradient*) ed è uno dei motivi per cui si preferiscono altre attivazioni, come la ReLU.

(b) Derivata di $\ln\alpha$. Con $\alpha = \sigma(z)$ e il risultato (a):

$$\frac{\partial \ln\alpha}{\partial z} = \frac{1}{\alpha}\cdot\frac{\partial\alpha}{\partial z} = \frac{1}{\alpha}\cdot\alpha(1-\alpha) = 1-\alpha$$

(equivalente a $\frac{e^{-z}}{1+e^{-z}} = 1-\alpha$).

(c) Derivata di $\ln(1-\alpha)$. Prima semplifichiamo l'argomento: $1-\alpha = 1-\frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$. Allora

$$\ln(1-\alpha) = \ln e^{-z} - \ln(1+e^{-z}) = -z + \ln\frac{1}{1+e^{-z}} = -z + \ln\alpha$$

e usando (b):

$$\frac{\partial \ln(1-\alpha)}{\partial z} = -1 + (1-\alpha) = -\alpha$$

(Stesso risultato per via diretta: $\frac{1}{1-\alpha}\cdot(-\alpha(1-\alpha)) = -\alpha$.)

(d) Derivata di z rispetto al peso. $z = w_0x_0 + w_1x_1 + \cdots + w_nx_n$ è lineare nei pesi; nella derivata rispetto a w_i sopravvive solo il termine w_ix_i:

$$\frac{\partial z}{\partial w_i} = x_i$$

(e) Un solo esempio. Sia $\mathrm{LL}_\ell = y\ln\alpha + (1-y)\ln(1-\alpha)$ il contributo dell'esempio ℓ (così $\mathrm{LL} = \sum_\ell \mathrm{LL}_\ell$). Per la regola della catena e (b), (c), (d):

$$\begin{aligned}\frac{\partial \mathrm{LL}_\ell}{\partial w_i} &= \frac{\partial \mathrm{LL}_\ell}{\partial z}\cdot\frac{\partial z}{\partial w_i} = \Big[y(1-\alpha) + (1-y)(-\alpha)\Big]x_i \\ &= \big[y - y\alpha - \alpha + y\alpha\big]x_i = (y-\alpha)\,x_i\end{aligned}$$

(i due termini $\pm y\alpha$ si cancellano).

(f) Somma su tutti gli esempi. La derivata di una somma è la somma delle derivate:

Gradiente della log-verosimiglianza

$$\frac{\partial \mathrm{LL}(\mathbf{w})}{\partial w_i} = \sum_{\ell=1}^{N} x_i^\ell\left(y^\ell - \alpha^\ell\right) \qquad \Big(\text{in forma vettoriale: } \nabla_{\mathbf{w}}\mathrm{LL} = X^T(\mathbf{y}-\boldsymbol{\alpha})\Big)$$

dove X è la matrice $N\times(n+1)$ con $X_{\ell i} = x_i^\ell$, $\mathbf{y} = (y^1,\ldots,y^N)^T$, $\boldsymbol{\alpha} = (\alpha^1,\ldots,\alpha^N)^T$.

■

Regola di aggiornamento completa. Sostituendo nel passo di discesa (ricordando $E = -\mathrm{LL}$):

$$w_i \leftarrow w_i + \eta\sum_{\ell=1}^{N} x_i^\ell\left(y^\ell - \alpha^\ell\right) \qquad \text{per ogni } i = 0,\ldots,n$$

dove ad ogni iterazione i α^ℓ vanno *ricalcolati* con i pesi correnti.

Come leggerlo: ogni esempio contribuisce con (**valore della feature**) $\times$ (**errore** = etichetta vera $-$ probabilità prevista). Se il modello è già corretto ($y \approx \alpha$) il contributo è nullo; altrimenti il peso viene spinto nella direzione che riduce l'errore. Si noti la semplicità: la derivata della sigmoide $\alpha(1-\alpha)$ si semplifica completamente con quella del logaritmo $\frac{1}{\alpha}$, ed è per questo che la cross-entropy è la loss «giusta» per la sigmoide.

1.5.3 Regolarizzazione e confronto con i modelli generativi

Algoritmo

Si parte da pesi nulli (o casuali) e si ripete fino a convergenza (es. finché l'accuracy sul test è soddisfacente):

$$w_i \leftarrow w_i + \eta \sum_{\ell=1}^{N} x_i^{(\ell)} \big(y^{(\ell)} - P(Y=1 \mid \mathbf{x}^{(\ell)}, \mathbf{w})\big)$$

In forma vettoriale (X matrice dei dati con colonna di 1 per il bias) l'intero ciclo è:

1. inizializza $\mathbf{w} \leftarrow \mathbf{0}$;
2. **ripeti**: calcola $\boldsymbol{\alpha} = \sigma(X\mathbf{w})$; calcola il gradiente $\mathbf{g} = X^T(\mathbf{y} - \boldsymbol{\alpha})$; aggiorna $\mathbf{w} \leftarrow \mathbf{w} + \eta\,\mathbf{g}$;
3. **fino a** che $\|\eta\,\mathbf{g}\| < \epsilon$ (incremento trascurabile), oppure l'accuracy su un insieme di validazione smette di migliorare, oppure si raggiunge un numero massimo di epoche.

Fermarsi quando l'errore su validazione risale (*early stopping*) è anche una forma implicita di regolarizzazione.

- modello corretto ($y \approx \alpha$): errore ≈ 0, il peso resta fermo;
- positivo sottostimato ($y = 1$, $\alpha \approx 0$): il peso cresce se $x_i > 0$, aumentando la probabilità;
- negativo sovrastimato ($y = 0$, $\alpha \approx 1$): il peso diminuisce.

Esempio: un passo di gradiente

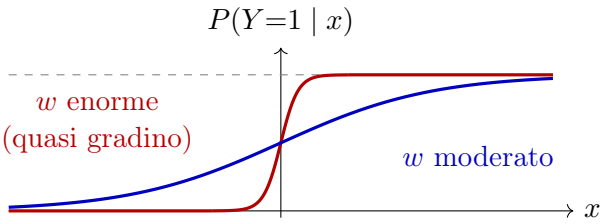
Un solo esempio $x = 2$, $y = 1$, pesi iniziali $w_0 = w_1 = 0$, $\eta = 0.1$.

- $z = 0 \Rightarrow \alpha = \sigma(0) = 0.5$, errore $y - \alpha = 0.5$.
- $\Delta w_0 = \eta \cdot 1 \cdot 0.5 = 0.05, \quad \Delta w_1 = \eta \cdot x \cdot 0.5 = 0.1 \cdot 2 \cdot 0.5 = 0.1.$
- Nuovi pesi $(0.05, 0.1)$: $z = 0.05 + 0.1 \cdot 2 = 0.25 \Rightarrow \alpha = \sigma(0.25) \approx 0.56$.

La probabilità della classe corretta è salita da 0.50 a 0.56. Ripetendo il passo continua a crescere.

Regolarizzazione

Se i dati sono perfettamente separabili da un iperpiano, la verosimiglianza continua a crescere aumentando i pesi: l'ottimizzatore li spinge a $\pm\infty$ per rendere la sigmoide un gradino e le probabilità esattamente 0 o 1. Pesi enormi = modello fragile e in **overfitting**.



Si aggiunge allora alla funzione obiettivo una **penalità** sui pesi (equivalente a una prior sui pesi), con $\lambda > 0$:

$$L_2 \text{ (weight decay)}: \quad w_i \leftarrow w_i - \eta\lambda w_i + \eta \sum_{\ell} x_i^{(\ell)}(y^{(\ell)} - \alpha^{(\ell)})$$

$$L_1 \text{ (Lasso)}: \quad w_i \leftarrow w_i - \eta\lambda \,\mathrm{sign}(w_i) + \eta \sum_{\ell} x_i^{(\ell)}(y^{(\ell)} - \alpha^{(\ell)})$$

- L_2 tiene i pesi **piccoli**; L_1 ne porta molti **esattamente a zero** (modello sparso, feature selezionate automaticamente);
- riduce la varianza e migliora la generalizzazione sui dati mai visti;
- λ regola il compromesso: troppo alto $\rightarrow$ underfitting, troppo basso $\rightarrow$ overfitting.

Perché L_1 azzera i pesi e L_2 no? Il termine di penalità sposta ogni peso verso 0 a *ogni* passo: con L_2 di una quantità *proporzionale* a w_i ($\eta\lambda w_i$), che si fa sempre più piccola avvicinandosi a 0, quindi w_i si avvicina a 0 senza mai raggiungerlo davvero; con L_1 di una quantità *costante* ($\eta\lambda$), che continua a spingere con la stessa forza fino ad arrivare a 0. Esempio con $w = 0.5$, $\eta\lambda = 0.1$ e senza altri termini: L_2 dà $0.5 \rightarrow 0.45 \rightarrow 0.405\ldots$, L_1 dà $0.5 \rightarrow 0.4 \rightarrow 0.3 \rightarrow 0.2 \rightarrow 0.1 \rightarrow 0$.
Un'altra lettura: L_2 equivale a una prior *gaussiana* centrata in 0 sui pesi, L_1 a una prior di *Laplace*; la regolarizzazione è la stima MAP al posto della MLE. Nella pratica λ si sceglie per validazione incrociata.

Modelli generativi vs discriminativi

Entrambi stimano $f : X \rightarrow Y$ o $P(Y \mid X)$, con filosofie opposte:

Confronto concettuale

- **Discriminativi** (es. regressione logistica): apprendono direttamente la **frontiera di decisione**; assumono una forma parametrica per $P(Y \mid X)$ e ne stimano i parametri.
- **Generativi** (es. Naïve Bayes): modellano la **distribuzione dei dati** ($P(X \mid Y)$ e $P(Y)$) e ricavano $P(Y \mid X)$ con Bayes. Possono anche *generare* campioni sintetici e rilevare anomalie (dati con $P(X)$ bassa).

	Generativo (Naïve Bayes)	Discriminativo (Reg. logistica)
Modella	$P(X, Y) = P(X \mid Y)P(Y)$	direttamente $P(Y \mid X)$
Training	conteggi, **immediato**	ottimizzazione iterativa (gradiente)
Ipotesi	forti (indipendenza condizionata)	deboli: nessuna su come sono distribuite le X
Feature correlate	le «conta due volte» e sbaglia le probabilità	i pesi si adattano alle correlazioni
Pochi dati	spesso migliore (meno parametri, converge prima)	rischia overfitting
Molti dati	limitato dalle ipotesi sbagliate	di solito migliore
Usi extra	generazione, anomaly detection	solo predizione

1.5.4 Modelli lineari in alta dimensione e multiclasse

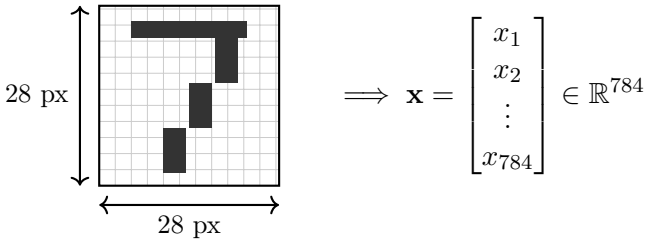
Si può pensare che i modelli lineari siano «troppo deboli». In realtà diventano molto efficaci con **molte feature**: in alta dimensione i punti sono **radi** (sparsi) e separare le classi con un iperpiano diventa facile. Intuizione: in 2D tre punti in posizione generica si separano in ogni modo possibile con una retta (tranne casi come XOR con 4 punti); con più dimensioni, aumentano i casi separabili.

Il caso di studio: MNIST

- Immagini in scala di grigio 28×28 di cifre manoscritte $0, \ldots, 9$.
- 60 000 immagini di training e 10 000 di test.
- Ogni pixel è una feature: $28 \times 28 = \mathbf{784}$ feature.



Esempi di MNIST: scrittura molto varia; i riquadri evidenziano cifre ambigue o scritte in modo anomalo.



Classificazione multiclasse e interpretazione visiva dei pesi

Regressione logistica multinomiale (Softmax)

Con 10 classi si impara un vettore di pesi $\mathbf{w}_k \in \mathbb{R}^{784}$ (più un bias) per ogni classe k. Ogni classe ha il suo punteggio $z_k = w_{k0} + \mathbf{w}_k \cdot \mathbf{x}$ e la **softmax** li trasforma in probabilità che sommano a 1:

$$P(Y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Con due classi si ritrova la sigmoide. Esempio con tre classi e punteggi $\mathbf{z} = (2, 1, 0.1)$:

$$e^z = (7.39,\ 2.72,\ 1.11), \quad \text{somma} = 11.21 \quad \Longrightarrow \quad P = (0.66,\ 0.24,\ 0.10)$$

Si predice la classe col punteggio più alto, come in Naïve Bayes. La softmax amplifica le differenze (l'esponenziale premia il punteggio massimo) ma conserva l'ordine.

Addestramento. Con etichetta *one-hot* $y_k^\ell \in \{0, 1\}$ (1 solo per la classe vera) la loss è la cross-entropy generalizzata $E = -\sum_\ell \sum_k y_k^\ell \ln P(Y{=}k \mid \mathbf{x}^\ell)$, e il gradiente ha la stessa forma del caso binario:

$$\frac{\partial \mathrm{LL}}{\partial w_{ki}} = \sum_\ell x_i^\ell \big(y_k^\ell - P(Y{=}k \mid \mathbf{x}^\ell)\big) \qquad \Longrightarrow \qquad \nabla_W \mathrm{LL} = X^T(Y - P)$$

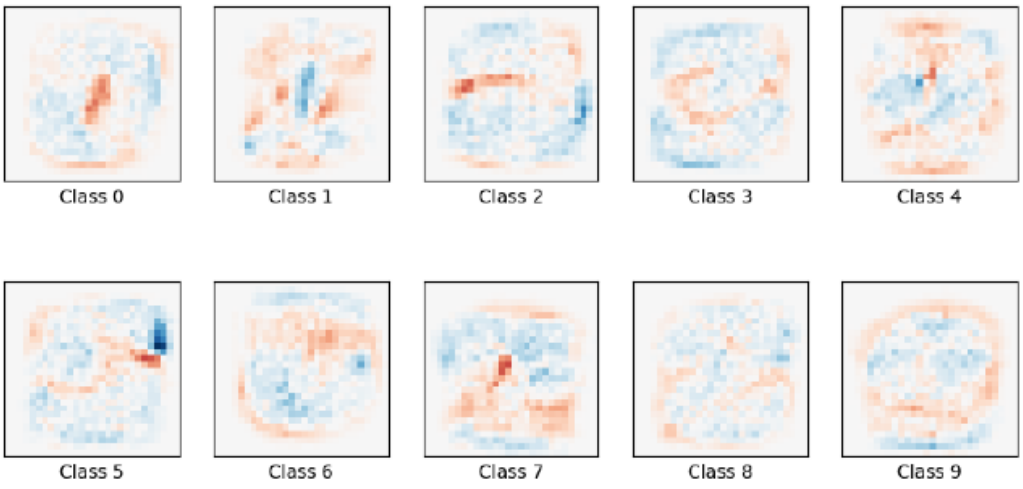
(feature × errore, per ogni classe). Con $K = 2$ si ritrova la regressione logistica binaria.
Con penalità L_1 molti pesi vanno esattamente a zero (sparsità). Risultati su MNIST:

- **sparsità**: 25.18% dei pesi azzerati;
- **accuratezza sul test set**: 92.57%;
- **tempo di addestramento**: circa 22.4 secondi.

Un modello lineare semplice, veloce e compatto supera il 92% su 10 classi.

I pesi come immagini

Il vettore $\mathbf{w}_k$ ha 784 componenti, una per pixel: può essere rimodellato in una griglia 28×28 e **visualizzato come immagine**. È una «maschera» della cifra k:



Pesi $\mathbf{w}_k$ delle 10 classi rimodellati a 28×28. Colori opposti = segno opposto.

Come leggere le immagini dei pesi

- **Pesi positivi**: pixel che, se accesi, *aumentano* la probabilità della classe.
- **Pesi negativi**: pixel che, se accesi, *riducono* la probabilità della classe (tratti incompatibili con quella cifra).
- **Pesi quasi nulli** (bianco): pixel irrilevanti, es. i bordi dell'immagine, sempre vuoti.

Le mappe non sono «sagome pulite» delle cifre: sono rumorose perché il modello non cerca di descrivere la cifra, ma solo di *discriminare* una classe dalle altre (approccio discriminativo), quindi pesa i pixel che distinguono, non quelli comuni a tutte.

Classificare = confrontare l'immagine con ogni maschera (prodotto scalare) e scegliere quella con cui c'è maggior **correlazione**: è la stessa idea del prodotto scalare tra documento e profilo di classe vista in Naïve Bayes.

Riepilogo: Regressione Logistica

- **Modello**: $P(Y = 1 \mid \mathbf{x}) = \sigma(w_0 + \mathbf{w} \cdot \mathbf{x})$; log-odds lineari, frontiera = iperpiano.
- **Training**: massimizzare la log-verosimiglianza (minimizzare la cross-entropy) per discesa del gradiente.
- **Gradiente**: $\partial \mathrm{LL}/\partial w_i = \sum_\ell x_i^{(\ell)}(y^{(\ell)} - \alpha^{(\ell)})$: feature $\times$ errore.
- **Loss convessa**: nessun minimo locale; il learning rate η va scelto con cura.
- **Regolarizzazione** L_1/L_2 contro l'overfitting e pesi esplosivi.
- **Multiclasse**: softmax, un vettore di pesi per classe.
- **Discriminativo**: modella $P(Y \mid X)$ senza assunzioni su X, a differenza di Naïve Bayes.

2 Reti neurali e deep learning

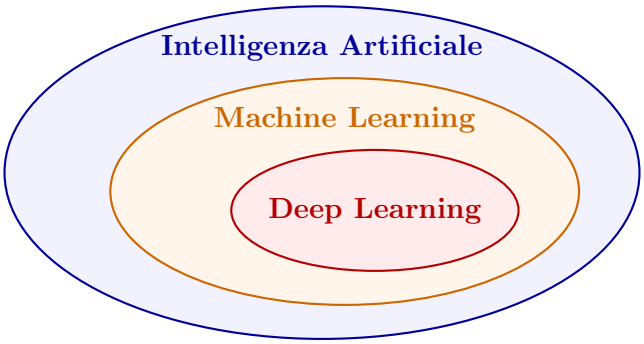
2.1 Deep Learning in breve

2.1.1 Che cos'è il Deep Learning e dove si applica

Il tema si introduce con una serie di domande e risposte, ciascuna accompagnata dal ragionamento che sta dietro.

Che cos'è il Deep Learning?

È un **ramo del Machine Learning**. Come visto nell'introduzione, l'intelligenza artificiale contiene il Machine Learning (apprendere dai dati invece di programmare regole a mano), e questo contiene a sua volta il Deep Learning.



Un modello di Deep Learning è dunque un modello di ML: ha parametri $\mathbf{w}$ che si stimano dai dati minimizzando una loss con la discesa del gradiente, esattamente come la regressione logistica del capitolo precedente. Cambia la *forma* del modello (molto più ricca), non il paradigma.

A che tipo di problemi si applica?

A **tutti i problemi adatti al Machine Learning**: classificazione, regressione, ... In generale, a ogni problema in cui esiste un fenomeno che si può descrivere con esempi $(\mathbf{x}, y)$ ma non con regole scritte a mano. Non è quindi una tecnica per un tipo di compito speciale: è un modo diverso di costruire il modello $f : X \to Y$.

Esistono problemi in cui eccelle in modo particolare?

Sì: i problemi con una **miriade di feature**, tipicamente dati non strutturati come immagini, audio/voce e testo, oppure sistemi di raccomandazione.

Dominio	Dato di ingresso	Feature	Ordine di grandezza
Immagini	matrice di pixel	un valore per pixel (e canale)	$28 \times 28 = 784$ (MNIST), $> 10^5$ per foto
Voce	forma d'onda campionata	ampiezza a ogni istante	decine di migliaia al secondo
Testo	sequenza di parole	presenza/identità di ogni parola	vocabolario da 10^4 a 10^5
Raccomandazione	storico utente-prodotto	interazione con ogni prodotto	milioni di prodotti

Perché proprio qui? Nei capitoli precedenti le feature erano poche e scelte a mano (es. *Outlook*, *Wind* negli alberi di decisione). Con migliaia di feature grezze (i singoli pixel) è impossibile progettarle a mano: servono modelli capaci di *costruire da soli* feature utili, combinando quelle elementari in livelli successivi (bordi → forme → oggetti). È proprio ciò che fanno le reti profonde. Il caso MNIST visto con la regressione logistica (784 feature, 92.6% di accuratezza con un modello *lineare*) mostra il punto di partenza; le reti neurali superano quel risultato proprio perché non sono limitate a combinazioni lineari dei pixel.

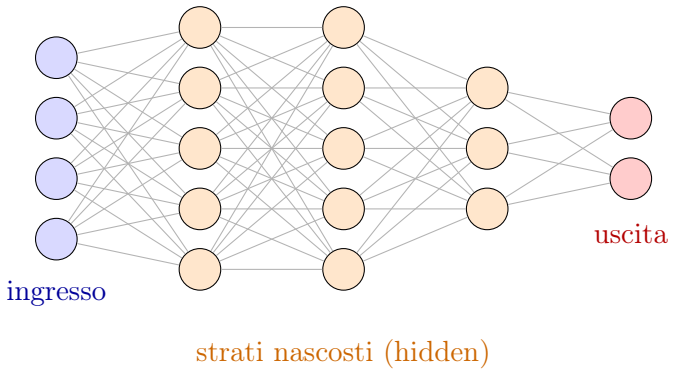
2.1.2 La tecnica di base: le reti neurali

Qual è la tecnica di base del DL?

Le **reti neurali** (Neural Networks).

Perché «Deep» (profondo)?

Perché sfrutta reti neurali **profonde**, composte da **molti strati (layer)** di neuroni. La profondità è il numero di strati che il dato attraversa dall'ingresso all'uscita.



Il singolo neurone è definito nella sezione successiva; anticipiamo l'idea: il neurone calcola $\sigma(w_0 + \mathbf{w} \cdot \mathbf{x})$, cioè *è esattamente una regressione logistica*. Una rete neurale è una composizione di tante regressioni logistiche, in cui le uscite di uno strato sono gli ingressi del successivo. Con un solo strato si ottiene un modello lineare; la profondità permette di rappresentare frontiere di decisione non lineari (come lo XOR, che un singolo iperpiano non separa).

Seconda risposta: le feature profonde. Il DL è «profondo» anche perché sfrutta **feature profonde** (*deep features*), cioè feature *estratte da altre feature*. Ogni strato prende in ingresso le uscite dello strato precedente e le ricombina: nel primo strato di una rete per immagini compaiono rilevatori di bordi, nei successivi forme, poi parti di oggetti, infine oggetti. Le feature non sono più progettate a mano, ma *apprese* insieme al classificatore.

2.2 Le reti neurali

2.2.1 Neurone artificiale, funzioni di attivazione e confronto col cervello

Rete neurale

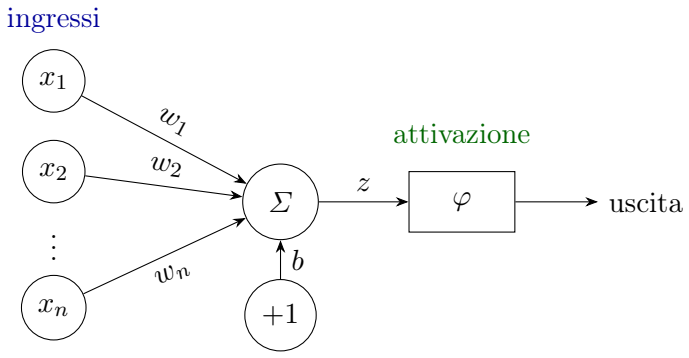
Una rete neurale è una **rete di neuroni (artificiali)**. Ogni neurone riceve più ingressi e produce una sola uscita, che può a sua volta essere ingresso di molti altri neuroni. In figura: uno strato di ingresso (*input layer*), due strati nascosti (*hidden layer*) e uno di uscita (*output layer*); gli strati nascosti sono quelli che non vediamo né in ingresso né in uscita.

Il neurone artificiale

Dati gli ingressi $x_1, \dots, x_n$, il neurone calcola

$$\text{uscita} = \varphi\Big(\underbrace{\sum_{i=1}^{n} w_i x_i + b}_{\text{somma pesata } z} \Big) = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$$

dove w_i sono i **pesi** (uno per ingresso), b è il **bias** (equivale a un ingresso fisso $+1$ con peso b, come lo w_0 visto in precedenza) e φ è la **funzione di attivazione**.



Punto chiave. Se $\varphi = \sigma$ (sigmoide), il neurone è esattamente la regressione logistica $\sigma(\mathbf{w} \cdot \mathbf{x} + b)$: stessi parametri, stessa formula. La novità delle reti sta nel *collegare molti neuroni*, non nel neurone. In generale la parte $\mathbf{w} \cdot \mathbf{x} + b$ è un modello lineare (come una regressione lineare); il neurone coincide con la regressione lineare se $\varphi(z) = z$ e con quella logistica se $\varphi = \sigma$.

Esempio: un neurone che calcola l'AND

Pesi $w_1 = w_2 = 1$, bias $b = -1.5$, attivazione a soglia (`if x>0 then 1 else 0`).

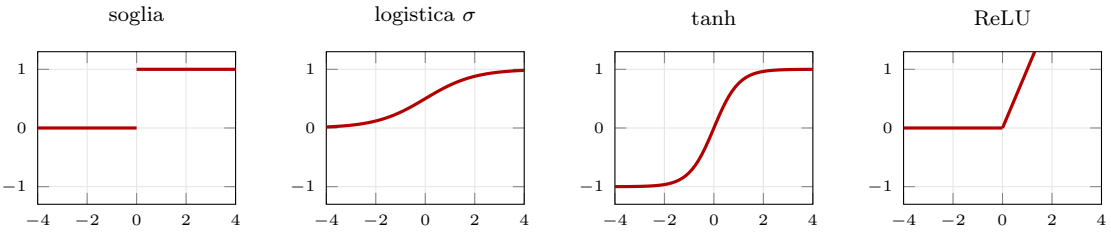
x_1	x_2	$z = x_1 + x_2 - 1.5$	uscita
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	$+0.5$	1

Con $b = -0.5$ si ottiene invece l'OR. Un singolo neurone non può però calcolare lo XOR (non è linearmente separabile): servono due strati, ad esempio $h_1 = \text{OR}(x_1, x_2)$, $h_2 = \text{AND}(x_1, x_2)$

e uscita $= 1$ se $h_1 - h_2 - 0.5 > 0$, cioè «OR ma non AND». È il motivo per cui servono gli strati nascosti.

Funzioni di attivazione

La funzione di attivazione è la responsabile dello «scatto» a soglia del neurone. Quattro scelte comuni:



Nome	Formula	Uscita	Note
Soglia	`if x>0 then 1 else 0`	$\{0, 1\}$	il neurone originale (percettrone); derivata nulla ovunque, quindi *non* addestrabile col gradiente
Logistica	$\dfrac{1}{1+e^{-x}}$	$(0, 1)$	soglia «morbida» e derivabile, interpretabile come probabilità; satura per $\lvert x\rvert$ grande
Tangente iperbolica	$\dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$(-1, 1)$	sigmoide centrata in 0 (è una logistica scalata: $\tanh x = 2\sigma(2x) - 1$)
ReLU	$\max(0, x)$	$[0, \infty)$	molto economica; non satura per $x > 0$, per cui attenua il problema del vanishing gradient

A cosa serve la funzione di attivazione

Serve a introdurre una **non linearità**, e questo è ciò che permette alla rete di avere un **potere espressivo** elevato. Vediamo perché con un ragionamento in tre passi.

Passo 1: senza attivazione, più strati valgono quanto uno. Prendiamo due strati *senza* funzione di attivazione (o, che è lo stesso, con $\varphi(z) = z$). Il primo calcola $\mathbf{h} = W_1\mathbf{x}$ e il secondo $\mathbf{y} = W_2\mathbf{h}$. Mettendo insieme i due calcoli:

$$\mathbf{y} = W_2(W_1\mathbf{x}) = \underbrace{(W_2W_1)}_{W}\,\mathbf{x} = W\mathbf{x}$$

Il prodotto di due matrici è ancora una matrice, quindi i due strati si *riducono* a uno solo. Lo stesso vale per tre, dieci o cento strati. Una rete profonda ma lineare è in realtà un modello lineare, esattamente come la regressione logistica o il Naïve Bayes visti prima: sa tracciare solo **rette** (iperpiani) per separare le classi.

Esempio numerico: due strati lineari si fondono

Con un solo ingresso x: primo strato $h = 2x$, secondo strato $y = 3h$. Allora $y = 3 \cdot (2x) = 6x$: è identico a un unico strato con peso 6. Aggiungere il secondo strato non ha dato nessuna capacità in più.

Passo 2: l'attivazione «rompe» la fusione. Se tra i due strati inseriamo una funzione non lineare φ, il calcolo diventa $\mathbf{y} = W_2\,\varphi(W_1\mathbf{x})$. Ora φ sta in mezzo e le due matrici *non* si possono più moltiplicare tra loro: ogni strato aggiunge davvero qualcosa. Per questo l'attivazione si applica in ogni strato nascosto, subito dopo la somma pesata.

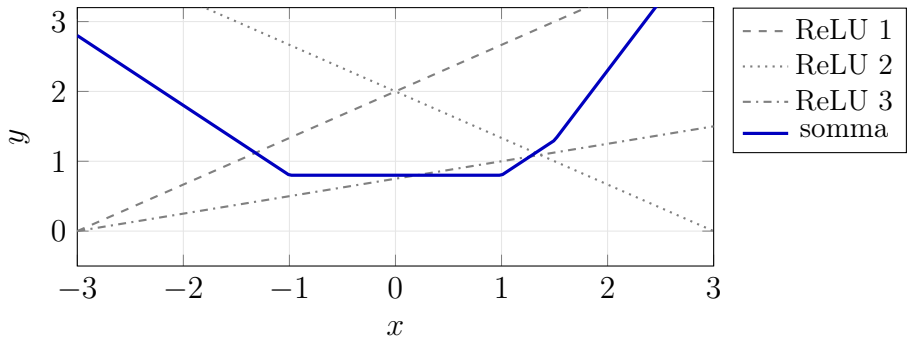
Esempio: lo stesso calcolo con una ReLU in mezzo

Riprendiamo $h = 2x$ e $y = 3h$, ma applichiamo ReLU a h: $y = 3 \cdot \max(0,\, 2x)$.

x	$h = \max(0, 2x)$	$y = 3h$
-2	0	0
-1	0	0
0	0	0
1	2	6
2	4	12

Il grafico non è più una retta: per $x < 0$ resta fermo a 0, per $x > 0$ sale come $6x$. Ha un «gomito», e una retta non potrà mai averlo.

Passo 3: più non linearità, più forme possibili. Ogni neurone con attivazione contribuisce con un piccolo «gomito» o una piccola curva. Sommando tanti gomiti si può approssimare una curva qualunque, un po' come ricostruire una linea curva usando tanti piccoli segmenti dritti. In figura, tre neuroni ReLU sommati danno una funzione a tratti con più pendenze diverse:



Le tre curve grigie sono i singoli neuroni (ognuno è una ReLU spostata), la curva blu è la loro somma pesata: una forma complicata, con tre cambi di pendenza, che nessuna retta può riprodurre. Con *molti* neuroni la curva può seguire quasi qualsiasi andamento. Questa idea ha un nome preciso: il **teorema di approssimazione universale** dice che una rete con uno strato nascosto sufficientemente largo e un'attivazione non lineare può approssimare (con la precisione che si vuole) qualunque funzione continua. Ha però un limite pratico: dice che la rete *esiste*, non come trovare i pesi giusti, né quanti neuroni servano; con più strati si ottengono di solito gli stessi risultati con molti meno neuroni.

In sintesi

- **Senza** attivazione: $W_2(W_1\mathbf{x}) = W\mathbf{x}$, la rete è un modello lineare, per quanti strati abbia.
- **Con** attivazione non lineare: ogni strato aggiunge capacità, e la rete può imparare frontiere di decisione curve (come per lo XOR).
- Più strati e più neuroni $\Rightarrow$ più **potere espressivo**, ma anche più parametri da stimare e più rischio di overfitting.

Come scegliere l'attivazione

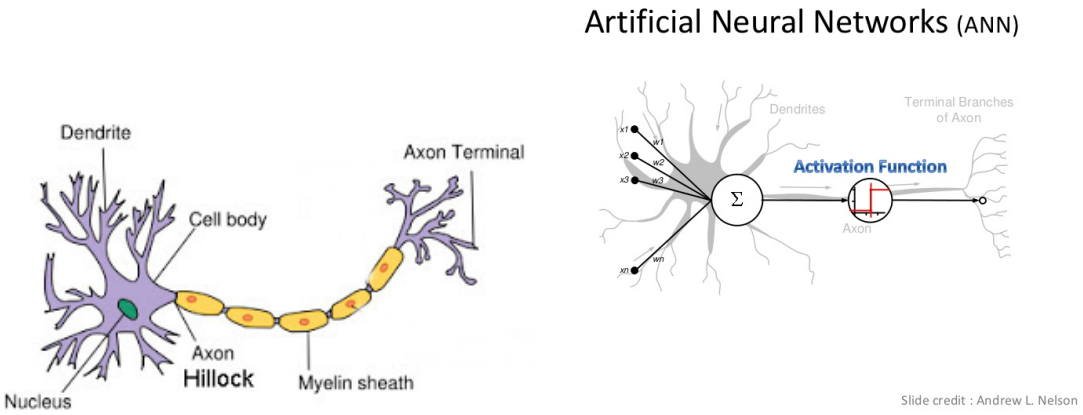
- **Strati nascosti**: di solito la **ReLU**, perché è semplice, veloce e non satura per valori positivi (il gradiente non si «spegne»).
- **Uscita per classificazione binaria**: la **sigmoide**, perché dà un numero in $(0, 1)$ interpretabile come probabilità.
- **Uscita per più classi**: la **softmax** (come nella regressione logistica multinomiale).
- **Uscita per regressione** (valore qualunque): nessuna attivazione, cioè $\varphi(z) = z$.
- La **soglia** non si usa nell'addestramento perché la sua derivata è zero quasi ovunque e il gradiente non avrebbe nulla da seguire.

Il passaggio dalla soglia alla sigmoide non è un dettaglio: l'addestramento a gradiente richiede funzioni *derivabili*, e la sigmoide è la versione liscia della soglia.

Il neurone biologico

Il neurone corticale ha un **albero dendritico** che raccoglie gli ingressi dagli altri neuroni, i quali vengono sommati nel corpo cellulare; quando si supera una **soglia di attivazione** il cono d'emergenza dell'assone (*Axon Hillock*) genera un impulso che viaggia lungo l'assone verso altri neuroni.

Neurone biologico		**Neurone artificiale**
dendriti (ingressi)	$\leftrightarrow$	ingressi x_i
forza della sinapsi	$\leftrightarrow$	pesi w_i
somma nel corpo cellulare	$\leftrightarrow$	Σ
soglia dell'Axon Hillock	$\leftrightarrow$	funzione di attivazione φ (e bias)
assone (uscita)	$\leftrightarrow$	uscita verso altri neuroni



Neurone corticale e il suo confronto con il neurone artificiale (slide credit: A. L. Nelson).

Qualche numero sul cervello umano

- neuroni: circa $2 \cdot 10^{10}$;
- tempo di commutazione di un neurone: circa 0.001 s (**lento**!);
- sinapsi per neurone: circa $10^{4–5}$;
- tempo per riconoscere un'immagine: circa 0.1 s.

Se un neurone impiega 1 ms, in 0.1 s ci stanno al più $0.1/0.001 = 100$ passi in sequenza: il cervello è quindi **poco profondo** (meno di 100 strati) e a **parallelismo altissimo**. Anche una rete artificiale è un calcolo a pochi strati ma molto «larghi», in cui tutti i neuroni di uno strato lavorano insieme (per questo le GPU sono così adatte).

Perché studiare il calcolo neurale

- **capire il cervello**, simulandone il funzionamento;
- esplorare un **paradigma di calcolo diverso**, lontanissimo dai linguaggi di programmazione tradizionali (non si scrivono istruzioni, si addestrano pesi);
- **risolvere problemi pratici** difficili con le tecniche algoritmiche, *anche se il cervello funziona in modo diverso*: le reti artificiali sono ispirate al cervello, non ne sono una copia fedele.

2.3 Topologie di rete e tipi di strato

2.3.1 Feed-forward, strati densi e strati convolutivi

Reti feed-forward e ricorrenti

Se il grafo della rete è **aciclico** si parla di rete **feed-forward**: l'informazione va solo dall'ingresso all'uscita. Se ci sono **cicli** la rete è **ricorrente** (l'uscita di un neurone può tornare a influenzare se stesso nel tempo; utili per le sequenze, ma non trattate nel corso).

Strati (layer)

In una rete feed-forward i neuroni sono organizzati in **strati**. Con **più di uno strato nascosto** la rete è **profonda** (*deep*); con uno solo (o nessuno) è **superficiale** (*shallow*). Con L strati la rete calcola una composizione di funzioni

$$f(\mathbf{x}) = f_L\big(\cdots f_2\big(f_1(\mathbf{x})\big)\cdots\big)$$

dove ogni f_k è lo strato k-esimo.

Lo strato come componente: tensori in ingresso e in uscita

Oltre al neurone, il mattone principale di una rete è lo **strato** (*layer*), che contiene i neuroni. Uno strato prende in ingresso e restituisce in uscita dei **tensori**, cioè array *multidimensionali*. Davanti a ogni strato bisogna quindi chiedersi: *che tensore ricevo in ingresso e che tensore mi serve in uscita?*

È come incastrare i pezzi del Lego: l'uscita di uno strato deve avere la forma (*shape*) che l'ingresso dello strato successivo si aspetta, altrimenti i componenti non combaciano.

Perché multidimensionali? Un'immagine 2D ha tipicamente anche la dimensione dei **canali** (ad es. RGB, 3 canali), che può variare da uno strato all'altro: l'ingresso è già un tensore 3D (altezza $\times$ larghezza $\times$ canali). Con un lotto (*batch*) di immagini si aggiunge una quarta dimensione.

Strato denso (fully connected)

Ogni neurone dello strato $k-1$ è collegato a **ciascun** neurone dello strato k. Per un singolo neurone con n ingressi

$$I^n \cdot W^n + B^1 = O^1$$

e vettorizzando si calcolano m uscite in parallelo con un'unica moltiplicazione matriciale:

$$I^n \cdot W^{n\times m} + B^m = O^m$$

(poi si applica l'attivazione elemento per elemento). Le dimensioni: I vettore riga $1\times n$, W matrice $n \times m$ (colonna j = pesi del neurone j), B vettore $1 \times m$.

Esempio: quanti parametri?

Immagine MNIST appiattita ($n = 784$) in uno strato denso con $m = 128$ neuroni: W ha $784 \cdot 128 = 100\,352$ pesi più 128 bias, cioè **100 480 parametri** in un solo strato.

Gli strati densi lavorano su ingressi **piatti** (non strutturati): l'**ordine** degli elementi in ingresso è **irrilevante**. Se si permutano allo stesso modo i pixel di tutte le immagini, la rete (riaddestrata) funziona uguale: lo strato non sa nulla di «pixel vicini».

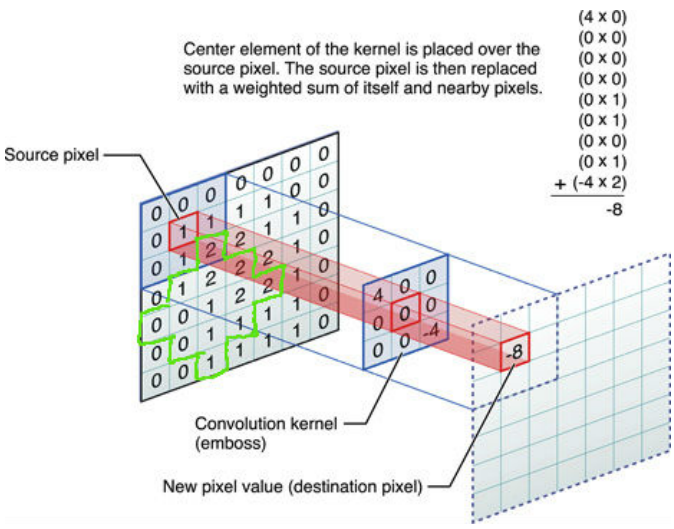
Strato convolutivo

Ogni neurone dello strato $k-1$ è collegato a un sottoinsieme *fisso e piccolo* di neuroni dello strato k tramite un **kernel** parametrico che viene fatto scorrere (*convoluto*) su tutto lo strato precedente:

1. si sposta il kernel K su una porzione M dell'ingresso di uguale dimensione;
2. si calcola il prodotto scalare $M \cdot K$ (ed eventualmente si somma un bias);

3. si sposta il kernel e si ripete.

La dimensione dell'uscita dipende solo dal *numero di volte* in cui si applica il kernel. L'ingresso è **strutturato** (le posizioni contano) e la struttura si riflette nell'uscita.



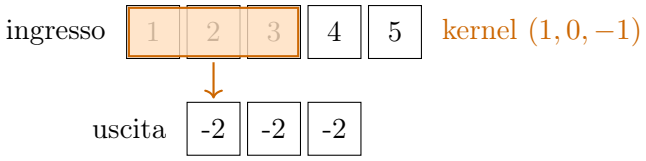
Convoluzione 2D: il kernel (qui un filtro *emboss*) centrato sul pixel sorgente produce il nuovo valore come somma pesata del pixel e dei suoi vicini.

Esempio 1D

Ingresso $\mathbf{x} = (1, 2, 3, 4, 5)$, kernel $K = (1, 0, -1)$ (lunghezza 3, passo 1):

$$(1, 2, 3) \cdot (1, 0, -1) = 1 - 3 = -2$$
$$(2, 3, 4) \cdot (1, 0, -1) = 2 - 4 = -2$$
$$(3, 4, 5) \cdot (1, 0, -1) = 3 - 5 = -2$$

Uscita $(-2, -2, -2)$, lunga $5 - 3 + 1 = 3$. Questo kernel misura la *variazione* locale (un rilevatore di bordi): su un ingresso costante darebbe 0, su una rampa dà un valore costante.



Perché è potente. Lo stesso kernel (stessi pesi) è riutilizzato in ogni posizione: un rilevatore di bordi serve ovunque nell'immagine. Il numero di parametri non dipende dalla dimensione dell'ingresso: un kernel 3×3 ha solo 9 pesi ($+1$ bias), contro i $784 \cdot 784 \approx 6 \cdot 10^5$ di uno strato denso che mantenga 784 uscite da un'immagine 28×28. Meno parametri, meno overfitting e struttura spaziale sfruttata.

	Denso	**Convolutivo**
Collegamenti	ogni neurone con tutti i precedenti	solo con un piccolo vicinato (kernel)
Pesi	uno per ogni coppia	condivisi in tutte le posizioni
Ingresso	piatto, ordine irrilevante	strutturato (immagini, serie temporali)
Uscita	dimensione scelta liberamente	dipende da quante volte si applica il kernel

2.4 Tuffo nel Deep Learning

2.4.1 Obiettivi, strumenti e storia

Un primo esempio

Un primo esempio pratico di rete addestrata (demo dal vivo a lezione).

Capire il DL

Gli obiettivi del corso su questo tema:

- capire i **diversi strati** e il loro scopo;
- capire come gli strati si organizzano in **architetture** rilevanti;
- capire le diverse **applicazioni** del DL e le soluzioni specifiche;
- capire i principali **problemi e costi** (dati, calcolo, overfitting).

Framework

TensorFlow/Keras (Google Brain), **PyTorch** (Facebook), **MXNet** (Apache). Fanno la differenziazione automatica (calcolano da soli i gradienti, quindi non si derivano a mano come fatto per la regressione logistica) e girano su GPU. Nel corso si usa principalmente **Keras**.

Cenni storici: l'eredità

Anno	Idea
1958	percettrone
1975	backpropagation
1980	strati convolutivi
1992	max-pooling
1997	LSTM

Progresso *estremamente lento*: le idee di base c'erano da decenni, ma mancavano dati e potenza di calcolo.

La rivoluzione del Deep Learning

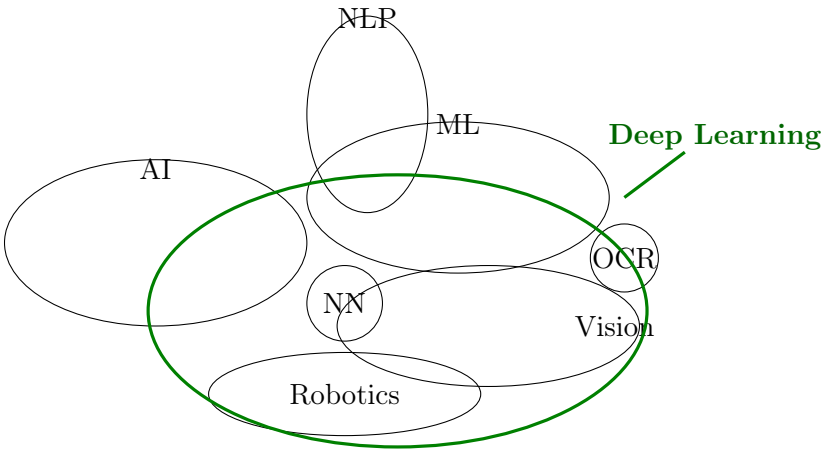
Dal 2011 in poi le pietre miliari si susseguono rapidamente:

2011	**Google Brain foundation**	2017	**PyTorch release**
2012	ReLU e Dropout	2017	Mask-RCNN
2012	ImageNet Competition	2017	PPO
2013	DQN	2018	Transformers
2014	Inception v1	2018	BERT
2015	**TensorFlow release**	2018	GPT
2015	Keras release	2018	Soft Actor Critic
2015	Batch normalization	2020	OpenAI Jukebox
2015	YOLO v1	2020	Vision Transformer
2015	**OpenAI foundation**	2021	**MXNet release**
2016	Residual connections	2022	DALL-E

Le cause della svolta: **più dati** (es. ImageNet), **più calcolo** (GPU), **migliori trucchi** (ReLU, Dropout, batch normalization, connessioni residuali) e **framework** aperti che hanno abbassato la barriera d'ingresso.

Il panorama a inizio secolo e nell'era del DL

All'inizio del 2000 i campi dell'IA erano separati ma con intersezioni: rappresentazione della conoscenza, sistemi esperti e programmazione a vincoli (AI «classica»); SVM, modelli bayesiani e clustering (ML); NLP, visione, OCR, robotica. Le reti neurali (NN) erano un'area piccola, all'incrocio fra questi campi.



Con la rivoluzione del DL il disegno cambia: le reti neurali profonde invadono quasi tutti gli altri campi (visione, NLP, robotica, OCR e parte del ML), diventando la tecnica dominante. Resta fuori buona parte della AI simbolica classica.

Riepilogo del capitolo

- **Neurone** $= \varphi(\mathbf{w} \cdot \mathbf{x} + b)$; con $\varphi = \sigma$ è una regressione logistica. Attivazioni: soglia, sigmoide, tanh, ReLU.
- Ispirazione biologica (dendriti $\rightarrow$ somma $\rightarrow$ soglia $\rightarrow$ assone) ma il cervello è poco profondo e molto parallelo.
- **Feed-forward** (aciclica) vs **ricorrente** (con cicli); **deep** se ha più di uno strato nascosto.
- **Strato denso**: tutti con tutti, $I \cdot W + B = O$, ingresso piatto. **Strato convolutivo**: kernel condiviso, ingresso strutturato, pochi parametri.
- Idee note dal 1958, ma la **rivoluzione** parte nel 2011–2012 grazie a dati, GPU, nuovi trucchi e framework (Keras, PyTorch, ...).